

(请大家预习)

第9章 数据聚类

Clustering Methods

张 燕 明

ymzhang@nlpr.ia.ac.cn

peopleucas.ac.cn/~ymzhang

模式分析与学习课题组(PAL)

多模态人工智能系统实验室 中科院自动化所

助教：杨 奇 (yangqi2021@ia.ac.cn)

张 涛 (zhangtao2021@ia.ac.cn)

上讲回顾

- 机器学习三要素

机器学习方法 = 模型(假设空间) + 评价(损失函数) + 优化算法

$$f^* = \min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n L(y_i, f(\mathbf{x}_i))$$

- 模型 $f(\mathbf{x})$ 的误差

- 训练(经验)误差

$$R_{emp}(f) = \frac{1}{n} \sum_{i=1}^n L(y_i, f(\mathbf{x}_i))$$

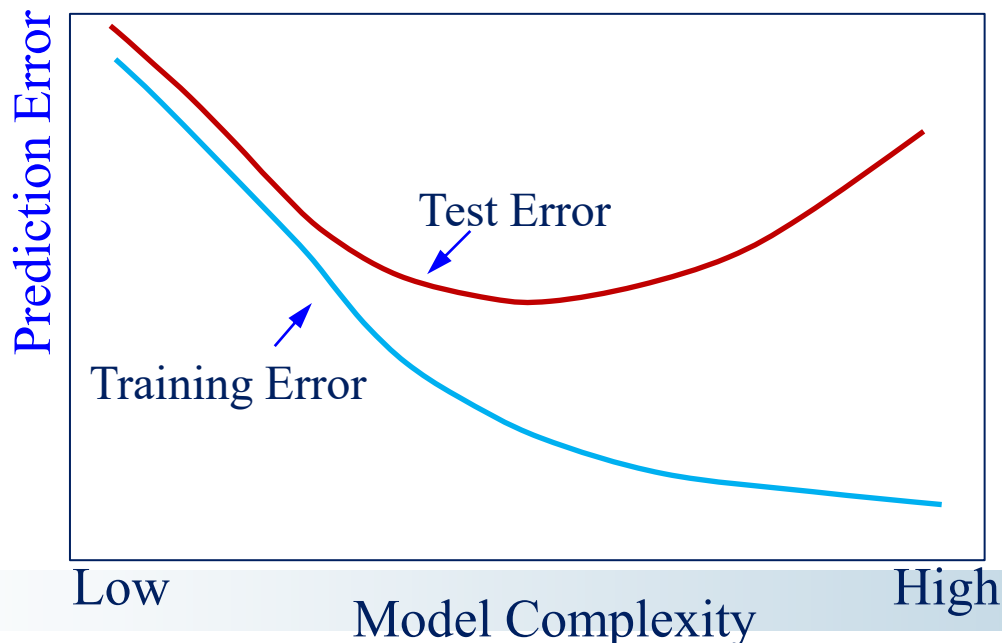
- 测试(泛化)误差

$$\begin{aligned} R_{exp}(f) &= E_{(\mathbf{X}, Y) \sim p(\mathbf{x}, y)} (L(Y, f(\mathbf{X}))) \\ &= \int L(y, f(\mathbf{x})) p(\mathbf{x}, y) d\mathbf{x} dy \end{aligned}$$

模型的测试误差不可知

上讲回顾

- 模型复杂度对误差的影响
 - 模型越复杂，对数据的拟合能力越强，训练误差越低
 - 但过分拟合训练数据会使模型关注样本的特性，而非数据的一般规律，从而导致测试误差增加，即过拟合
 - 由于“训练误差 $\neq$ 测试误差”，在评价和选择模型时，不能只考虑训练误差



上讲回顾

- 两种常见模型选择方法

- 正则化(结构风险最小化):

$$\min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n L(y_i, f(\mathbf{x}_i)) + \lambda J(f)$$

- 使用验证集:

- 样本划分: holdout, bootstrap, cross-validation等

上讲回顾

- 集成学习 (Ensemble Learning)
 - 目的：将若干单个分类器组合起来，共同完成最终的分类任务，期望取得比单个分类器更好的性能。
“三个臭皮匠顶个诸葛亮”
 - 问题：(1) 如何训练多个分类器；(2) 如何集成它们
 - 典型方法：
 - Stacked Generalization
 - Random Subspace
 - Bagging
 - Boosting

上讲回顾

Algorithm 1 Adaboost .

Input:

training dataset $(X, Y) = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}, y_i \in \{-1, +1\}$;
iterators: M ;

Output:

- 1: initialize each sample weight: $D_{1,i} = \frac{1}{N}, i = 1, 2, \dots, N$, and $f_0(x) = 0$
 - 2: **for** $m = 1$ to M **do**
 - 3: training a base learner: $G_m(x)$
 - 4: calculate error ratio: $e_m = \sum_{i=1}^N D_{m,i} I(G_m(x_i) \neq y_i)$
 - 5: if $e_m > 0.5$ then break
 - 6: $\alpha_m = \frac{1}{2} \ln(\frac{1-e_m}{e_m})$
 - 7: update sample weight: $D_{m+1,i} = \frac{D_{m,i}}{Z_m} \cdot \exp\{-\alpha_m y_i G_m(x_i)\},$
 $Z_m = \sum_{i=1}^N D_{m,i} \cdot \exp\{-\alpha y_i G_m(x_i)\}$
 - 8: $f_m(x) = f_{m-1}(x) + \alpha_m G_m(x)$
 - 9: **end for**
 - 10: **return** $G(x) = \text{sign}(f_M(x)) = \text{sign}(\sum_{m=1}^M \alpha_m G_m(x)).$
-

定理3: Adaboost算法是一种学习模型为加法模型、损失函数为指数函数、学习方法为前向分步算法的两类分类学习方法。

引言

- 聚类

- 物以类聚，人以群分。
- 将数据分成多个类别，同一个类内的样本之间具有较高的相似性，不同类样本间差异性较大。
- 对一批没有类别标签的样本集，按照样本之间的相似程度分类，相似的归为一类，不相似的归为其它类。这种分类称为聚类分析，也称为无监督分类。
- 聚类的质量(或结果)取决于对度量标准的选择。

引言

- 聚类任务

- 给定一个样本集合 $X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$ ，以及一种度量样本间相似度或者相异度(距离)的标准。聚类方法输出样本集 X 的一个划分，即 $D = \{D_1, D_2, \dots, D_K\}$ 。其中， D_i ($i = 1, 2, \dots, K$) 是 X 的一个子集，且满足：

- $D_1 \cup D_2 \cup \dots \cup D_K = X$

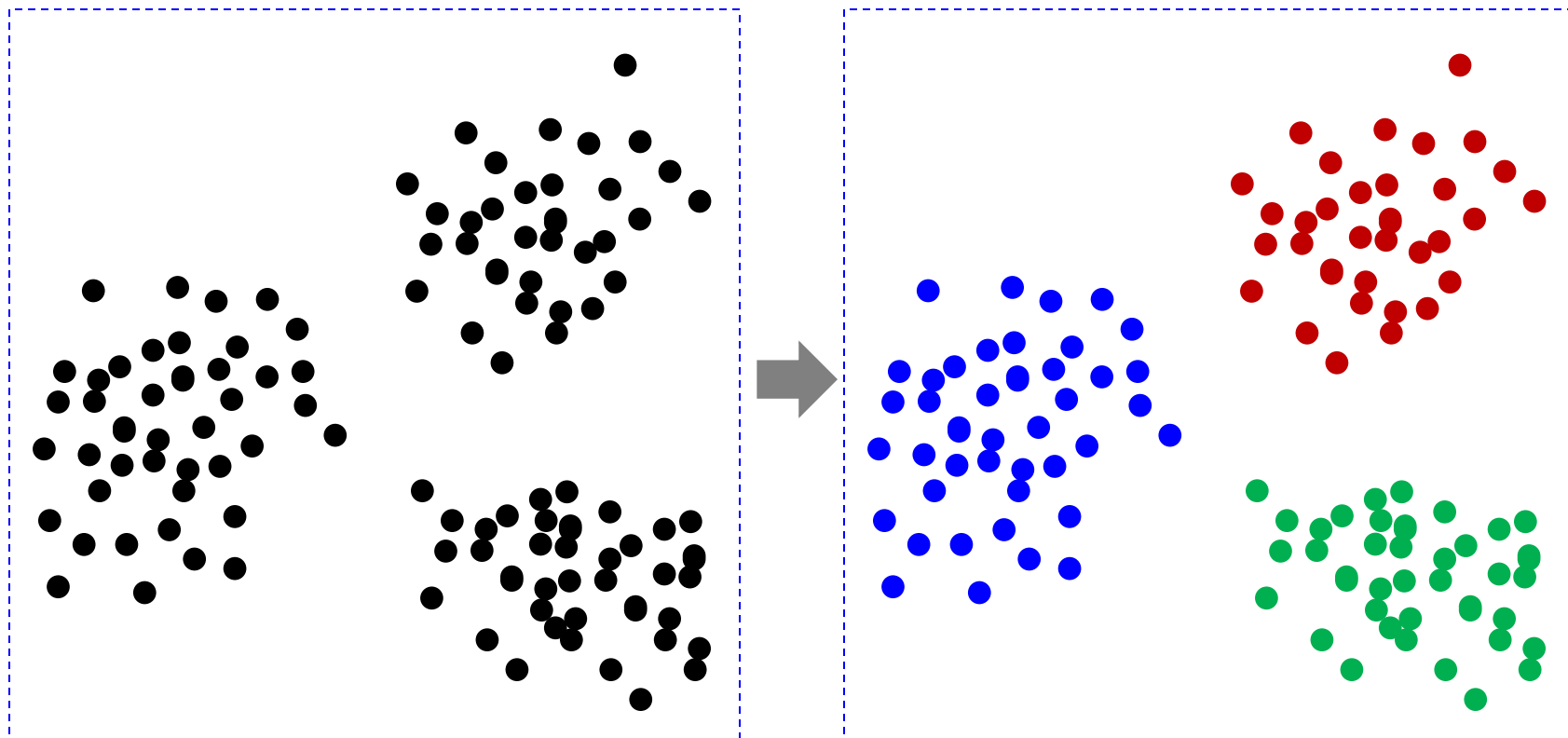
- $D_i \cap D_j = \emptyset, i \neq j$

- D 中成员 $D_1, D_2, \dots, D_K$ 叫做类或者簇(cluster)，每个类均通过一些特征来描述：

- 通过类中心或者类的边界点来表示；
 - 使用聚类树等图形化方式来表示。

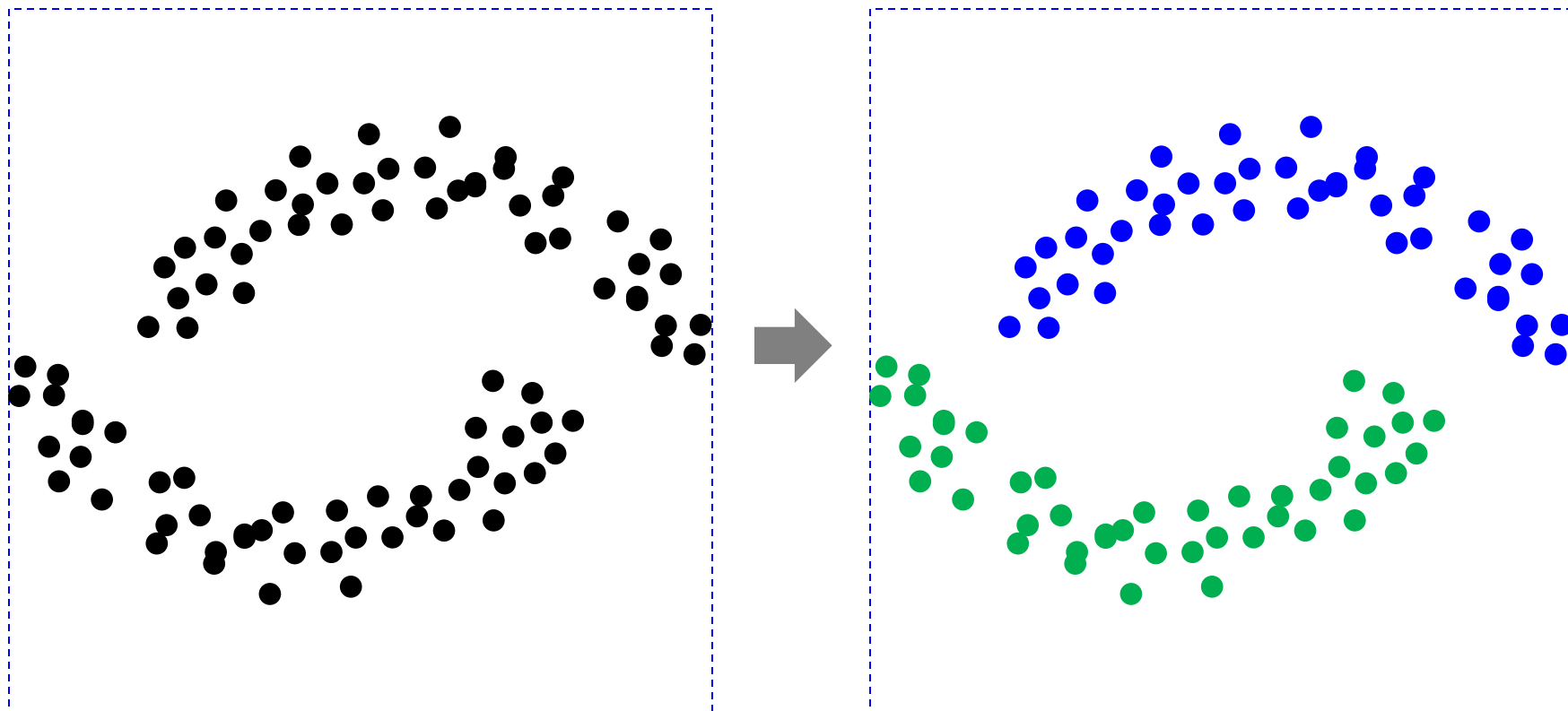
引言

- 例子



引言

- 例子



引言

- 聚类方法分类

- 按照度量准则

- 基于**距离**的聚类方法：基于各种不同的距离或者相似性来度量点对之间的关系，如**K-means**, **层次聚类**等。
 - 基于**概率密度**的聚类方法：采用密度函数对样本进行描述，并得到聚类结果，如**GMM**, **mean-shift**, **DBSCAN**等。
 - 基于**连通性**的聚类方法：主要包含基于图的方法。高度连通的数据通常被聚为一簇，如**谱聚类**等。

引言

- 聚类方法分类

- 按照不同的技术路线

- **模型法(原型聚类)**：为每一个簇引入一个模型，然后对数据进行划分，使其满足各自分派的模型，如 **K-means**。
 - **层次法**：对给定样本进行层次划分，如 **层级聚类**。
 - **密度法**：对数据的密度进行评价，如 **混合高斯模型**、**Mean-Shift**。
 - **网格法**：将数据空间划分为有限个单元网络结构，然后基于网络结构进行聚类，如 **矢量量化**。

引言

• 挑战性问题

— 对初始输入参数鲁棒

- 具有自适应的簇数判定能力（一直没有解决好）。
- 对初始聚类中心、近邻阈值、近邻个数等参数鲁棒。

— 能够发现任意形状的聚类

- 能够发现任意形状的簇，球状的、位于同一流形上的数据。因此，选择合适的距离度量很关键。

— 可伸缩性

- 可伸缩性是指聚类算法无论对于小数据集还是大数据集，都应有效；无论对小类别数据还是大别类数据，都应有效。

引言

- 挑战性问题

- 具有不同类型的数据处理能力

- 既可处理数值型数据，也可处理非数值型数据；既可处理离散数据，也可处理连续域内的数据。比如布尔型、时序型、枚举型、以及这些类型的混合。

- 能够处理高维数据

- 既可处理属性较少的数据，也可处理属性较多的数据。
 - 在高维空间聚类更具挑战性，对于高维稀疏数据，这一点更突出。

引言

- 挑战性问题

- 对噪声鲁棒

- 在实际中，绝大多数样本集都包含噪声、空缺、部分未知属性、孤立点、甚至错误数据。

- 具有约束的聚类

- 在实际应用中，通常需要在某种约束条件下进行聚类，既满足约束条件，以希望有高聚类精度，是一个挑战性问题。

距离与相似性度量

- 距离

- 设有 d 维空间的三个样本 $\mathbf{x}$, $\mathbf{y}$ 和 $\mathbf{z}$, 记 $d(\cdot, \cdot)$ 为一个 $\mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ 的映射, 如满足如下几个条件则称 $d(\cdot, \cdot)$ 为一个距离:

- $d(\mathbf{x}, \mathbf{y}) \geq 0$

非负性

- $d(\mathbf{x}, \mathbf{x}) = 0$

自相似性

- $d(\mathbf{x}, \mathbf{y}) = d(\mathbf{y}, \mathbf{x})$

对称性

- $d(\mathbf{x}, \mathbf{y}) \leq d(\mathbf{x}, \mathbf{z}) + d(\mathbf{z}, \mathbf{y})$

三角不等式

- 距离可以描述对点间的相异程度, 距离越大, 两个点越不相似; 距离越小, 两个点越相似。

距离与相似性度量

- 设 $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$, **Minkowski 距离度量**定义如下:

$$d(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^d |x_i - y_i|^q \right)^{\frac{1}{q}}$$



$q=1$	$d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^d x_i - y_i $	城区距离 曼哈顿距离
-------	--	---------------

$q=2$	$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^d x_i - y_i ^2}$	欧氏距离
-------	---	------

$q=\infty$	$d(\mathbf{x}, \mathbf{y}) = \max_{1 \leq i \leq d} x_i - y_i $	切比雪夫距离
------------	--	--------

距离与相似性度量

- 设 $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$, **Mahalanobis (马氏)距离**定义如下:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{(\mathbf{x} - \mathbf{y})^T \mathbf{M} (\mathbf{x} - \mathbf{y})}$$

其中, $\mathbf{M}$ 是半正定矩阵。

- $\mathbf{M}$ 为单位矩阵时, 退化为欧氏距离度量。
- $\mathbf{M}$ 为对角矩阵时, 退化为**特征加权**欧氏距离

距离与相似性度量

- 相似性

- 设 $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$, 余弦相似度定义如下:

$$s(\mathbf{x}, \mathbf{y}) = \left(\frac{\mathbf{x}}{\|\mathbf{x}\|} \right)^T \frac{\mathbf{y}}{\|\mathbf{y}\|} = \frac{\sum_{i=1}^d x_i y_i}{\sqrt{\sum_{i=1}^d x_i^2} \sqrt{\sum_{i=1}^d y_i^2}}$$

(两个模长为1的向量之内积)

距离与相似性度量

- 集合相似性

- 假设全集 S 包含 d 个元素，则 $\mathbf{x} \in \{0,1\}^d$ (即每维特征只取 $\{0,1\}$ 中的一个值) 可以表示 S 的一个子集， $x_i=1$ 表示包含第 i 个元素， $x_i=0$ 表示不包含
- 设 $\mathbf{x}, \mathbf{y} \in \{0,1\}^d$ ，为定义子集之间的相似性，先计算如下几个值：

- f_{00} : 样本 $\mathbf{x}$ 和 $\mathbf{y}$ 中满足 $x_i=y_i=0$ 的属性的个数
- f_{10} : 样本 $\mathbf{x}$ 和 $\mathbf{y}$ 中满足 $x_i=1 \& y_i=0$ 的属性的个数
- f_{01} : 样本 $\mathbf{x}$ 和 $\mathbf{y}$ 中满足 $x_i=0 \& y_i=1$ 的属性的个数
- f_{11} : 样本 $\mathbf{x}$ 和 $\mathbf{y}$ 中满足 $x_i=y_i=1$ 的属性的个数

- 进一步，可定义如下几种类型的相似性度量：

距离与相似性度量

- 集合相似性

- **简单匹配系数**(simple matching coefficient, SMC):

$$s_{SMC}(\mathbf{x}, \mathbf{y}) = \frac{f_{00} + f_{11}}{f_{00} + f_{10} + f_{01} + f_{11}}$$

- **Jaccard 相似系数**:

$$s_J(\mathbf{x}, \mathbf{y}) = \frac{f_{11}}{f_{10} + f_{01} + f_{11}} \quad \leftarrow \frac{|A \cap B|}{|A \cup B|}$$

- **Tanimoto 系数**:

$$s_T(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x}^T \mathbf{y}}{\mathbf{x}^T \mathbf{x} + \mathbf{y}^T \mathbf{y} - \mathbf{x}^T \mathbf{y}} = \frac{f_{11}}{\mathbf{x}^T \mathbf{x} + \mathbf{y}^T \mathbf{y} - f_{11}}$$

$\mathbf{x}^T \mathbf{x}$ 取1的个数 $\mathbf{y}^T \mathbf{y}$ 取1的个数

距离与相似性度量

- 举例：计算如下**两位顾客x和y**的相似度

商品	面包	啤酒	牛奶	咖啡	茶叶	鸡蛋	猪肉	牛肉	洋葱	土豆	大米	白糖
x	1	1	1	1	0	0	0	1	1	1	0	1
y	1	0	1	0	1	1	0	0	0	1	1	1

商品	莲藕	花生	可乐	豆腐	菠菜	黄瓜	面粉	酱油	辣椒	白酒	黄鱼	茄子
x	0	0	1	0	1	1	1	0	1	1	1	0
y	1	0	1	1	1	1	1	1	0	0	1	0

距离与相似性度量

- 类间距离

- **最短距离法**：定义两个类中最近的两个样本的距离为类间距离。

$$d(D_a, D_b) = \min \{d(\mathbf{x}, \mathbf{y}) \mid \mathbf{x} \in D_a, \mathbf{y} \in D_b\}$$

- **最长距离法**：定义两个类中最远的两个样本的距离为类间距离。

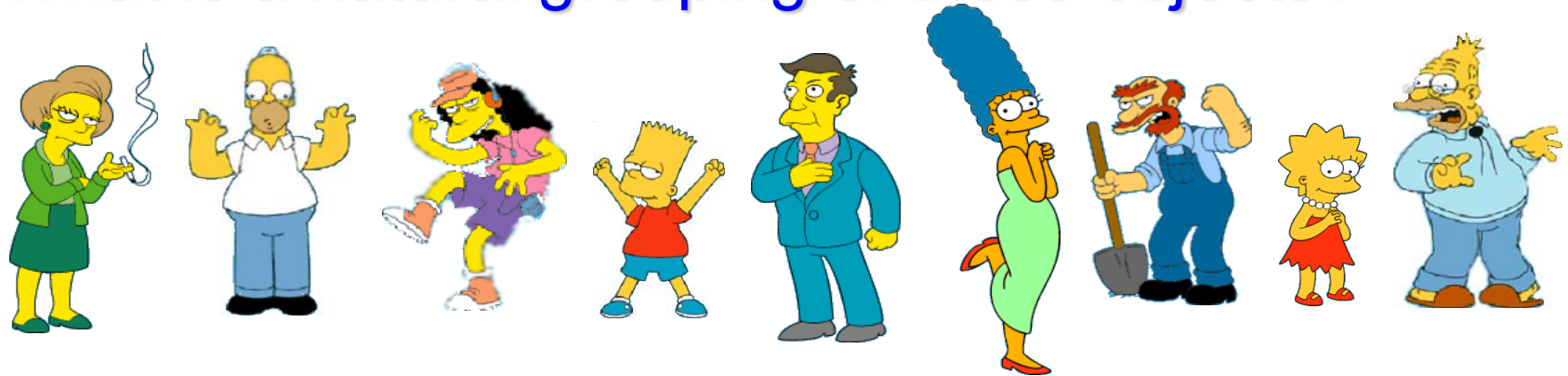
$$d(D_a, D_b) = \max \{d(\mathbf{x}, \mathbf{y}) \mid \mathbf{x} \in D_a, \mathbf{y} \in D_b\}$$

- **类直径**：类直径反映类中样本之间的差异，可定义为类中各样本至**类中心点**的平均欧氏距离：

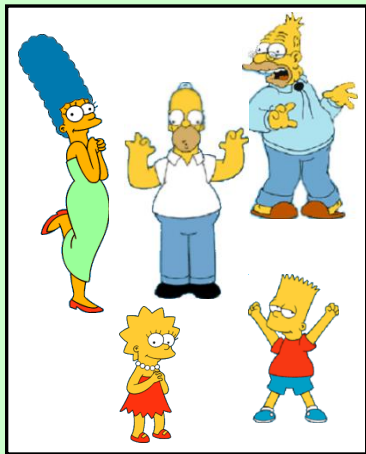
$$r(D_a) = \frac{1}{|D_a|} \sum_{\mathbf{x} \in D_a} \|\mathbf{x} - \bar{\mathbf{x}}\|^2$$

距离对聚类的影响

What is a natural grouping of these objects?



Clustering is subjective



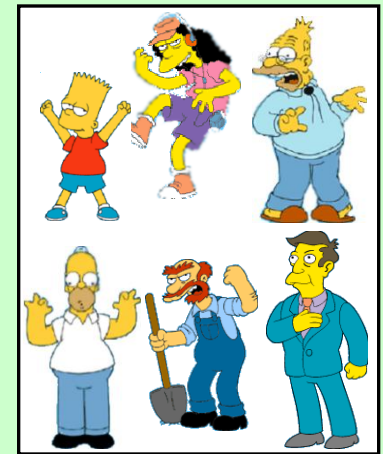
Simpson's Family



School Employees



Females



Males



身份识别 vs 姿态估计

***K*-means Clustering**

K -means algorithm (Lloyd, 1957)

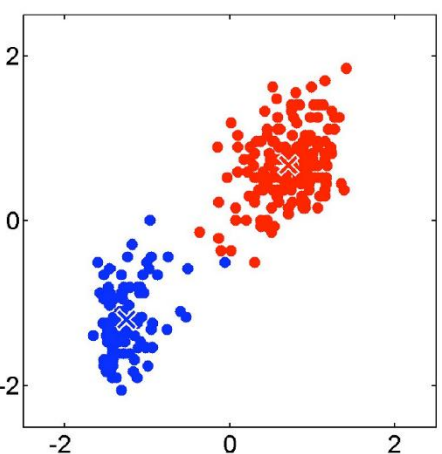
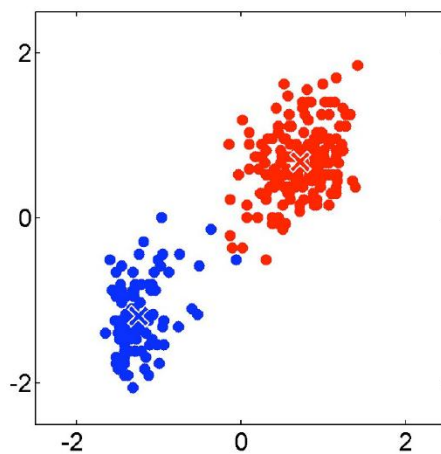
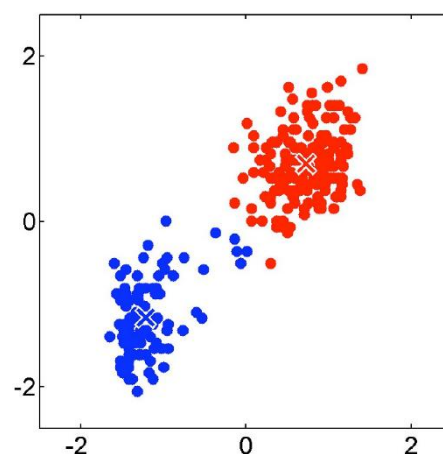
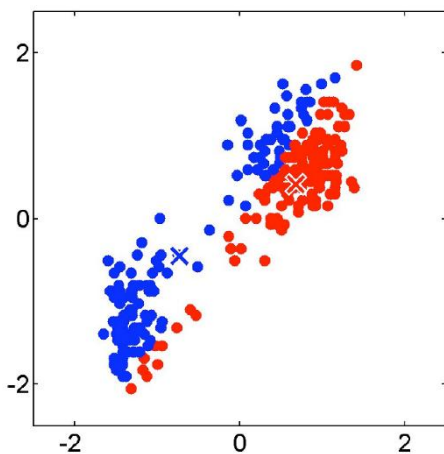
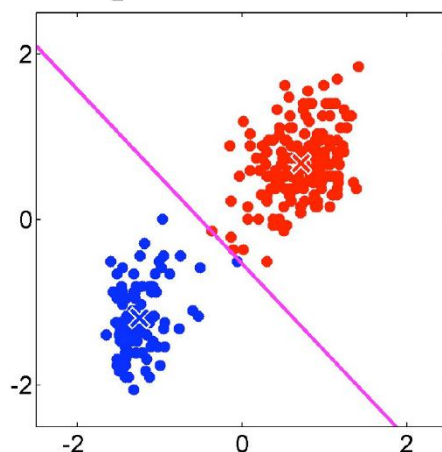
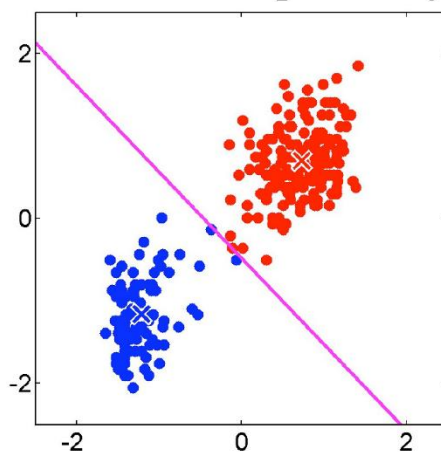
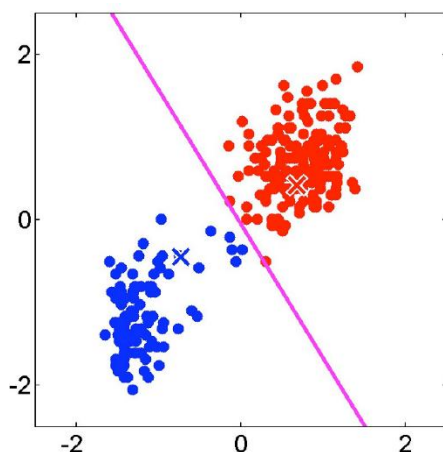
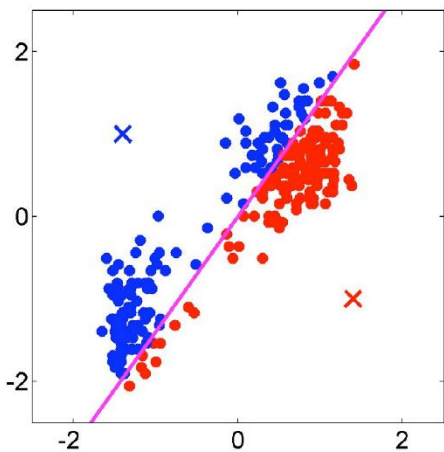
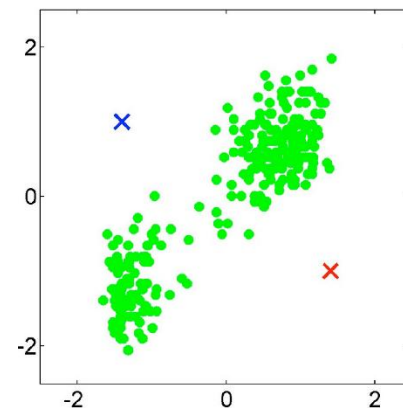
- **Input:** N examples $\{\mathbf{x}_1, \dots, \mathbf{x}_N\}$; $\mathbf{x}_n \in \mathbb{R}^D$; the number of partitions K
- **Initialize:** K cluster means $\mu_1, \dots, \mu_K$, each $\mu_k \in \mathbb{R}^D$
 - Usually initialized randomly, but good initialization is crucial; many smarter initialization heuristics exist (e.g., K -means++, Arthur & Vassilvitskii, 2007)
- **Iterate:**
 - (Re)-Assign each example $\mathbf{x}_n$ to its closest cluster center (based on the smallest Euclidean distance)

$$\mathcal{C}_k = \{n : k = \arg \min_k \|\mathbf{x}_n - \mu_k\|^2\}$$

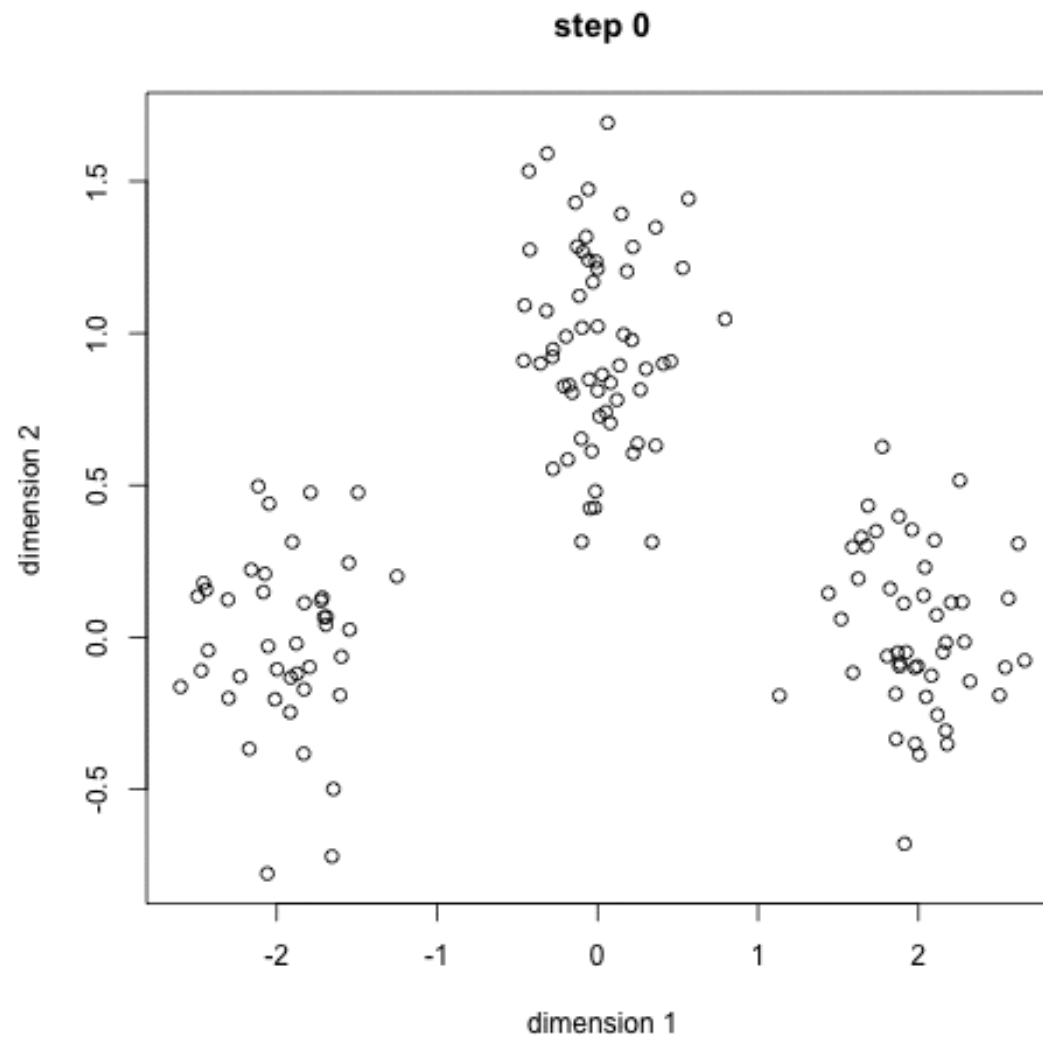
($\mathcal{C}_k$ is the set of examples assigned to cluster k with center μ_k)
 - Update the cluster means

$$\mu_k = \text{mean}(\mathcal{C}_k) = \frac{1}{|\mathcal{C}_k|} \sum_{n \in \mathcal{C}_k} \mathbf{x}_n$$
 - Repeat while not converged
- Stop when cluster means or the “loss” does not change by much

K -means algorithm



K -means illustration



Loss function of K -means?

- Let $\mu_1, \dots, \mu_K$ be the K cluster centroids (means)
- Let $z_{nk} \in \{0, 1\}$ be s.t. $z_{nk} = 1$ if $\mathbf{x}_n$ belongs to cluster k , and 0 otherwise
 - Note: $\mathbf{z}_n = [z_{n1} \ z_{n2} \ \dots \ z_{nK}]$ represents a length K one-hot encoding of $\mathbf{x}_n$
- Define the distortion or “loss” for the cluster assignment of $\mathbf{x}_n$

$$\ell(\mu, \mathbf{x}_n, \mathbf{z}_n) = \sum_{k=1}^K z_{nk} \|\mathbf{x}_n - \mu_k\|^2$$

- Total distortion over all points defines the K -means “loss function”

$$L(\mu, \mathbf{X}, \mathbf{Z}) = \sum_{n=1}^N \sum_{k=1}^K z_{nk} \|\mathbf{x}_n - \mu_k\|^2 = \|\mathbf{X} - \mathbf{Z}\mu\|^2$$

where $\mathbf{Z}$ is $N \times K$ (row n is $\mathbf{z}_n$) and μ is $K \times D$ (row k is μ_k)

- The K -means **problem** is to minimize this objective w.r.t. μ and $\mathbf{Z}$
 - Note that the objective only minimizes within-cluster distortions

K -means objective

- Consider the K -means objective function

$$L(\boldsymbol{\mu}, \mathbf{X}, \mathbf{Z}) = \sum_{n=1}^N \sum_{k=1}^K z_{nk} \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2$$

- It is a **non-convex** objective function
 - Many local minima possible
- Also **NP-hard** to minimize in general (note that $\mathbf{Z}$ is discrete)
- The **K -means algorithm** we saw is a heuristic to optimize this function
- K -means algorithm alternated between the following two steps
 - Fix $\boldsymbol{\mu}$, minimize w.r.t. $\mathbf{Z}$ (assign points to closest centers)
 - Fix $\mathbf{Z}$, minimize w.r.t. $\boldsymbol{\mu}$ (recompute the center means)

Convergence of K -means

- Each step (updating $\mathbf{Z}$ or μ) can never increase the objective
- When we update $\mathbf{Z}$ from $\mathbf{Z}^{(t-1)}$ to $\mathbf{Z}^{(t)}$

$$L(\mu^{(t-1)}, \mathbf{X}, \mathbf{Z}^{(t)}) \leq L(\mu^{(t-1)}, \mathbf{X}, \mathbf{Z}^{(t-1)})$$

because the new $\mathbf{Z}^{(t)} = \arg \min_{\mathbf{Z}} L(\mu^{(t-1)}, \mathbf{X}, \mathbf{Z})$

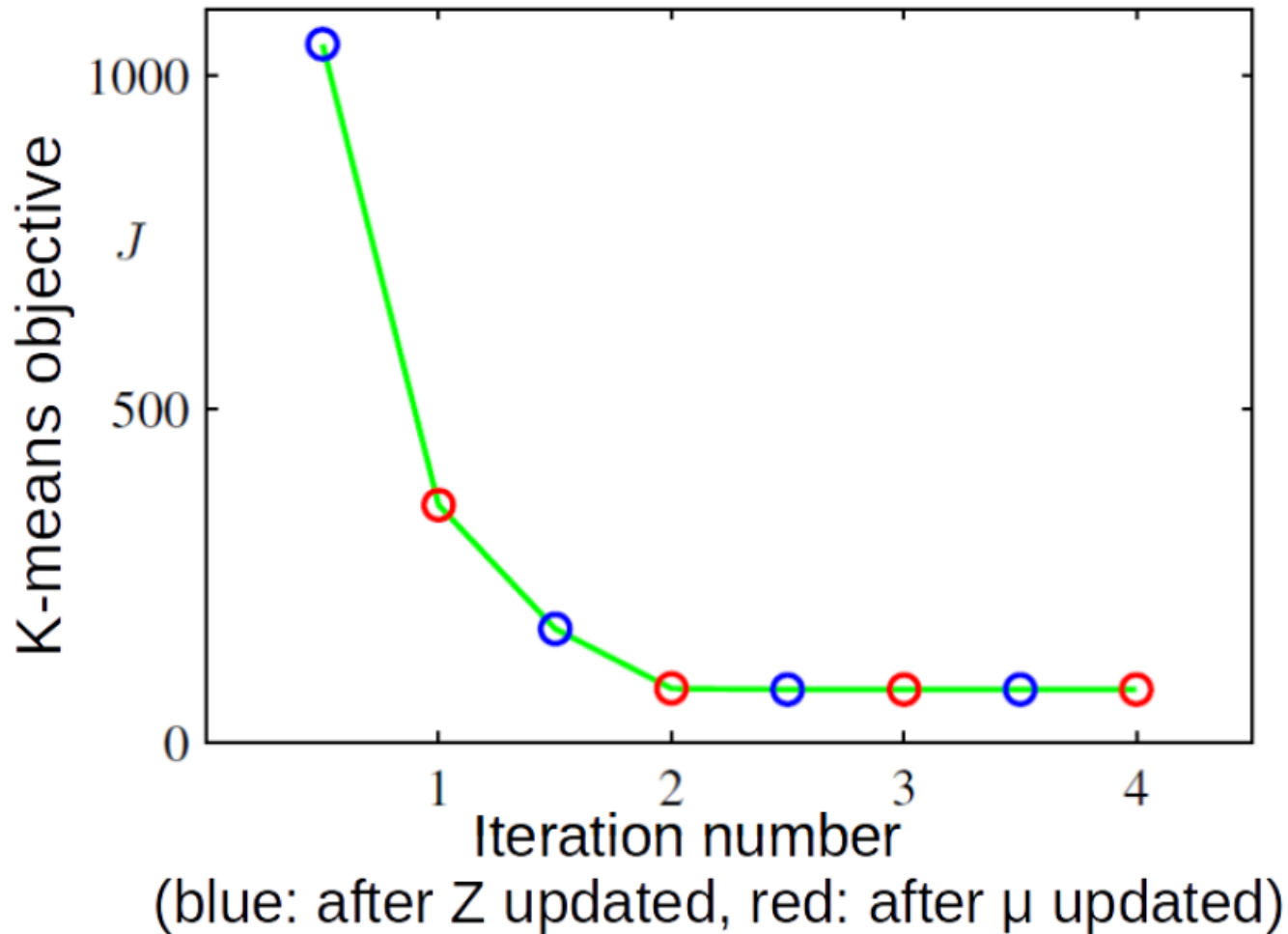
- When we update μ from $\mu^{(t-1)}$ to $\mu^{(t)}$

$$L(\mu^{(t)}, \mathbf{X}, \mathbf{Z}^{(t)}) \leq L(\mu^{(t-1)}, \mathbf{X}, \mathbf{Z}^{(t)})$$

because the new $\mu^{(t)} = \arg \min_{\mu} L(\mu, \mathbf{X}, \mathbf{Z}^{(t)})$

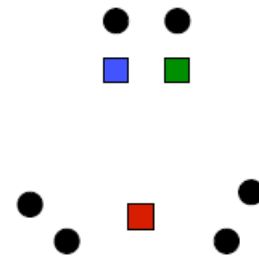
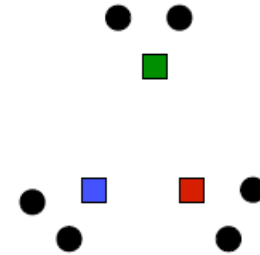
- Thus the K -means algorithm monotonically decreases the objective

Convergence of K -means



Local optimum: initialization matters

- **K-means is a heuristic algorithm**
 - Requires initial means
 - It does matter what you pick
 - What can go wrong?
 - Various schemes for preventing this kind of thing: variance-based split/merge, initialization heuristics

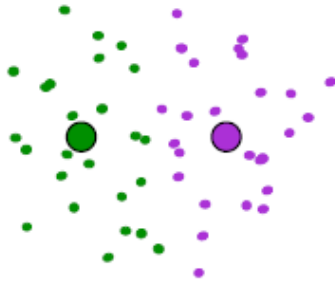


K-means++

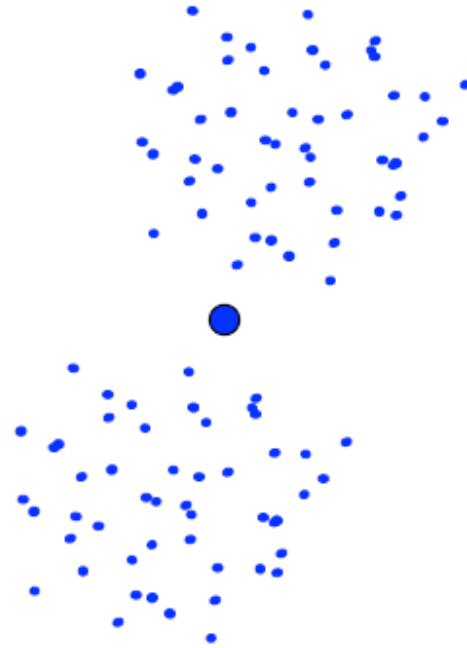
- How to avoid bad local minima?
- K-means++: **carefully select the initial means**
 - 1 Select μ_1 uniformly at random from $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$
 - 2 for $k = 2, \dots, K$:
 - 3 for $i = 1, \dots, n$:
 - 4 compute $d_i = \min \{\|\mathbf{x}_i - \mu_j\|, j = 1, \dots, k-1\}$
 - 5 end
 - 6 select μ_k from $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$ with prob. proportional to d_i
 - 7 end
 - 8 Output $\{\mu_1, \mu_2, \dots, \mu_K\}$ as the initial means

Local optimum: K matters

A local optimum:



Would be better to have
one cluster here



... and two clusters here

Running time of K -means

- Guaranteed to converge in a finite number of iterations
- Running time per iteration:
 1. Assign data points to closest cluster center
 $O(KN)$ time
 2. Change the cluster center to the average of it assigned points
 $O(N)$ time

Example: K -means for segmentation

Original



$K=10$



$K=3$



$K=2$



17%



8%



4%

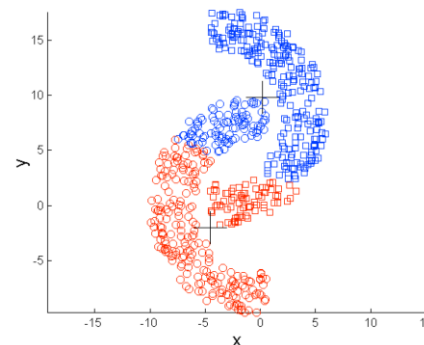
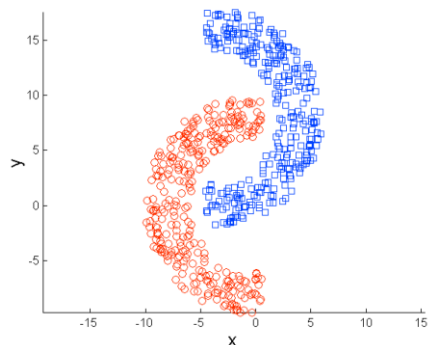
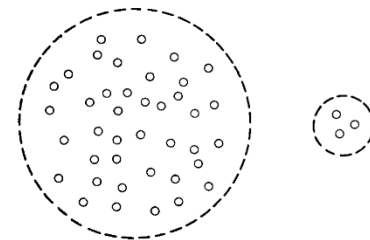
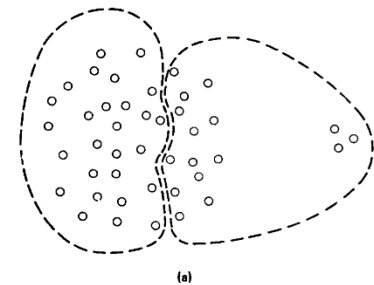
Example: vector quantization



FIGURE 14.9. *Sir Ronald A. Fisher (1890 – 1962) was one of the founders of modern day statistics, to whom we owe maximum-likelihood, sufficiency, and many other fundamental concepts. The image on the left is a 1024×1024 grayscale image at 8 bits per pixel. The center image is the result of 2×2 block VQ, using 200 code vectors, with a compression rate of 1.9 bits/pixel. The right image uses only four code vectors, with a compression rate of 0.50 bits/pixel*

Limitations of K -means

- Makes **hard assignments** of points to clusters
 - A point either completely belongs to a cluster or doesn't belong at all
 - No notion of a **soft assignment** (i.e., **probability** of being assigned to each cluster)
some point $\mathbf{x}_n$, $p_1 = 0.7$, $p_2 = 0.2$, $p_3 = 0.1$
- Works well only if the clusters are **roughly of equal sizes**
- Probabilistic clustering methods such as **Gaussian mixture models** can handle both these issues (model each cluster using a Gaussian distribution)
- K -means also works well only when the clusters are **round-shaped** and does badly if the clusters have **non-convex shapes**



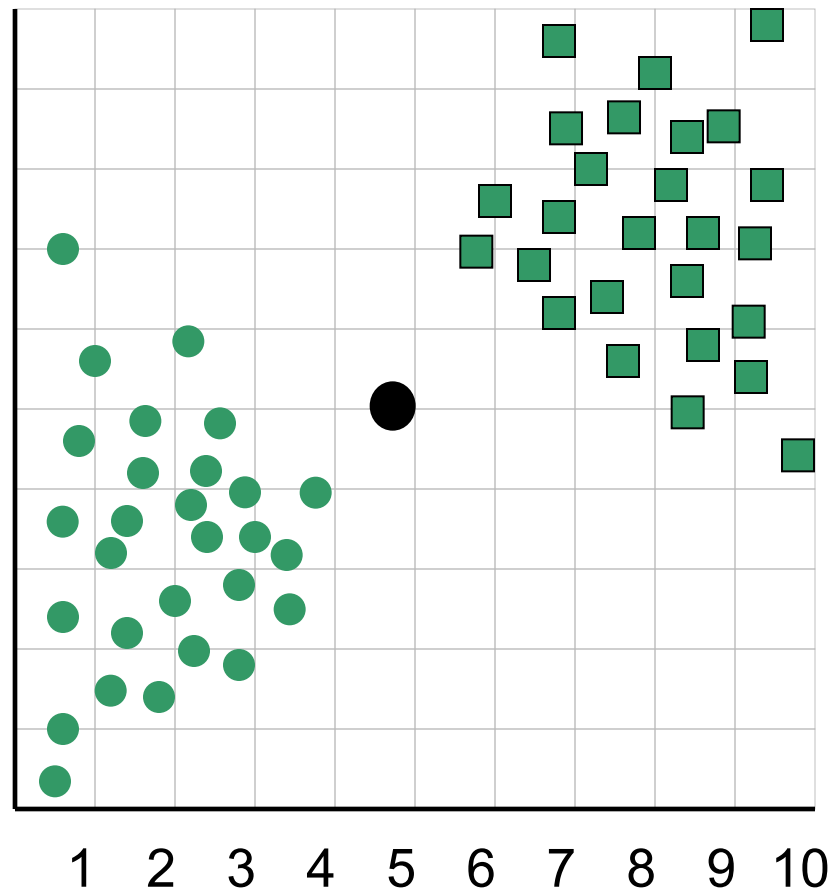
- **Kernel K -means** or **Spectral clustering** can handle non-convex

Selection of K

- Idea 1: Use trick of cross validation to select K
 - What should we optimize?
 - Problem?
- Idea 2: Let domain expert look at the clustering and decide if they like it
 - How should we show this to them?
 - Problem?
- Idea 3: The “knee” solution

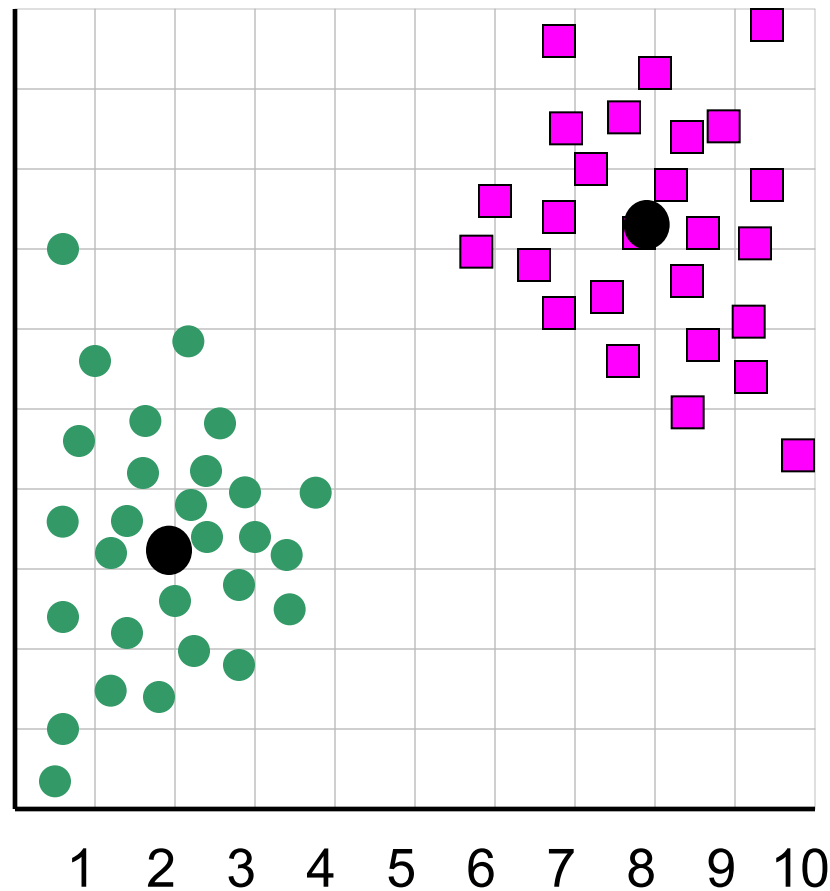
Selection of K

When $K = 1$, the objective function is 873.0



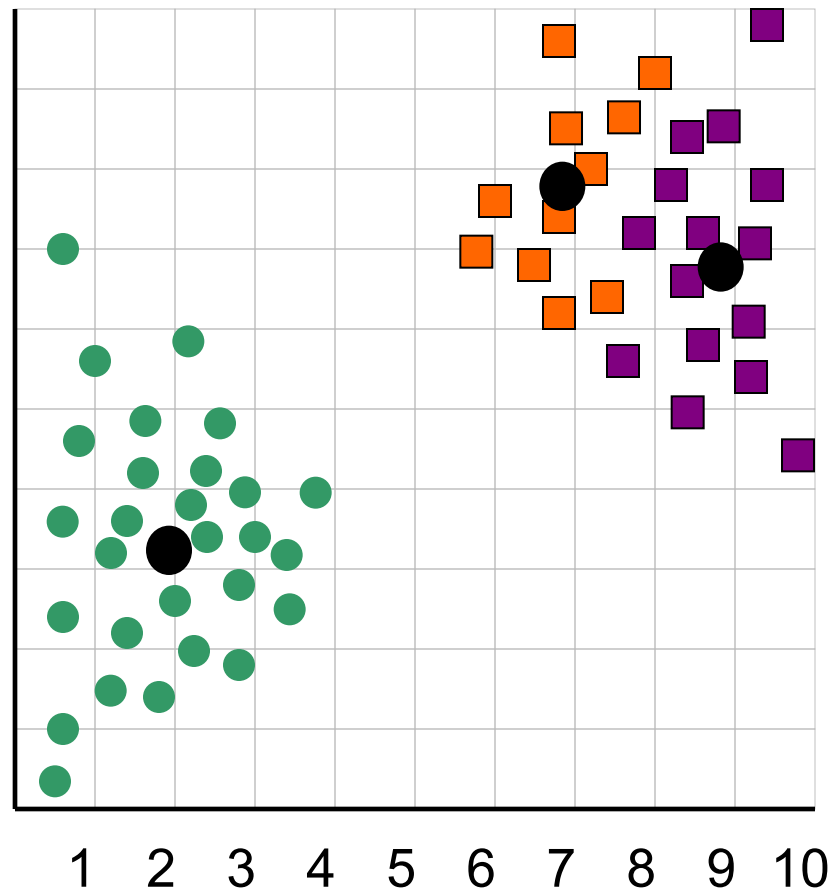
Selection of K

When $K = 2$, the objective function is 173.1



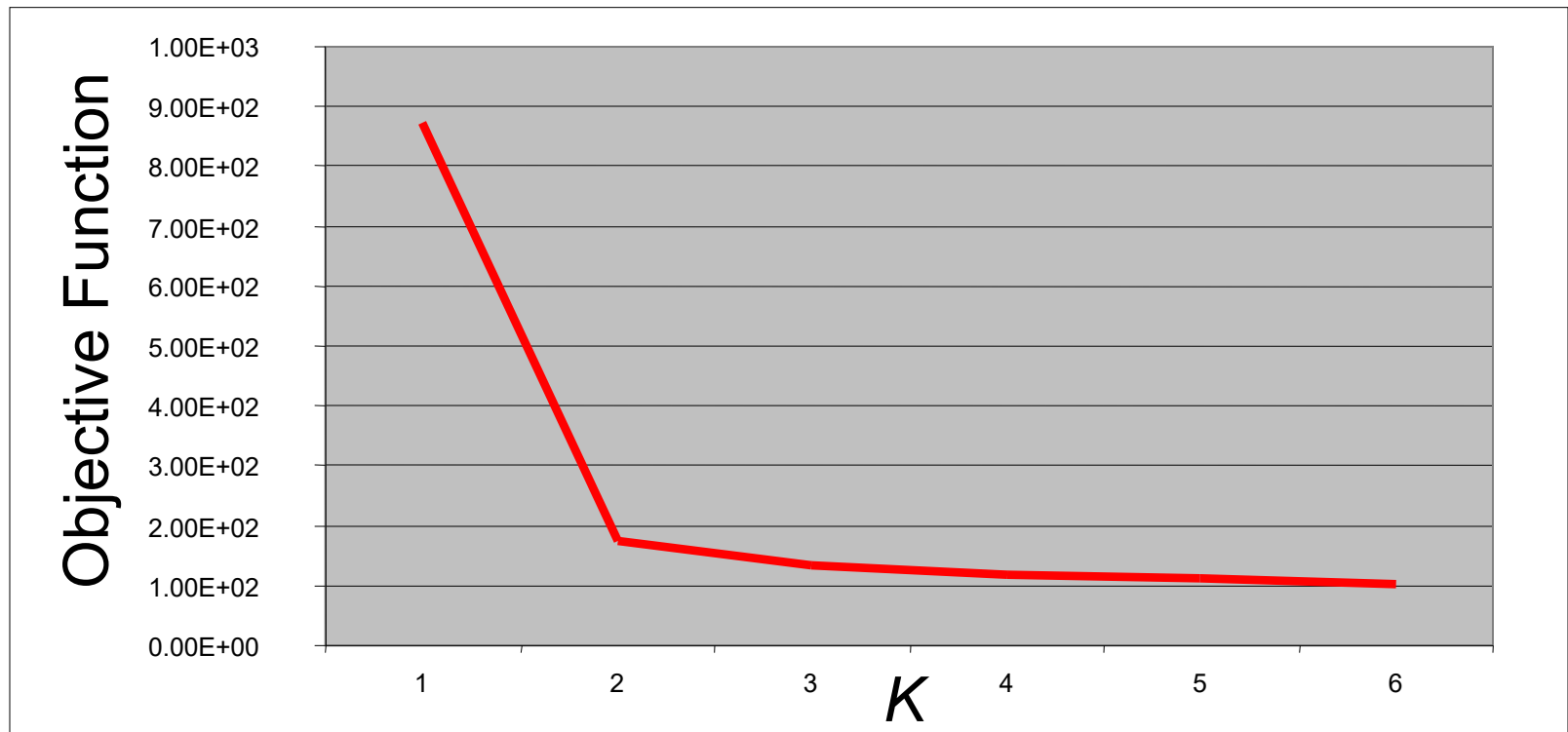
Selection of K

When $K = 3$, the objective function is 133.6



We can plot the objective function values for k equals 1 to 6...

The abrupt change at $K = 2$, is highly suggestive of two clusters in the data. This technique for determining the number of clusters is known as “knee finding” or “elbow finding”.



High-dimensional K -means

- Difficult to find true clusters
 - Irrelevant and redundant features
 - All points are equally close
- Solutions: Dimension Reduction
 - PCA, LDA, CCA, ICA
 - Feature selection
 - Cluster ensembles using random projection

K-Medians

- **K-Medians** is another clustering algorithm related to **K-Means**.
- Instead of recomputing the group center points using the mean, we use the **median vector** of the group.
- This method is **less sensitive to outliers** (because of using the Median)
- But is much slower for larger datasets as sorting is required on each iteration when computing the median vector.

K-Medians

- **Assignment step**: assigns each observation to the cluster with the closest center.
- **Update step**: calculates the **new median of each feature of each cluster** to be the new center of that cluster.

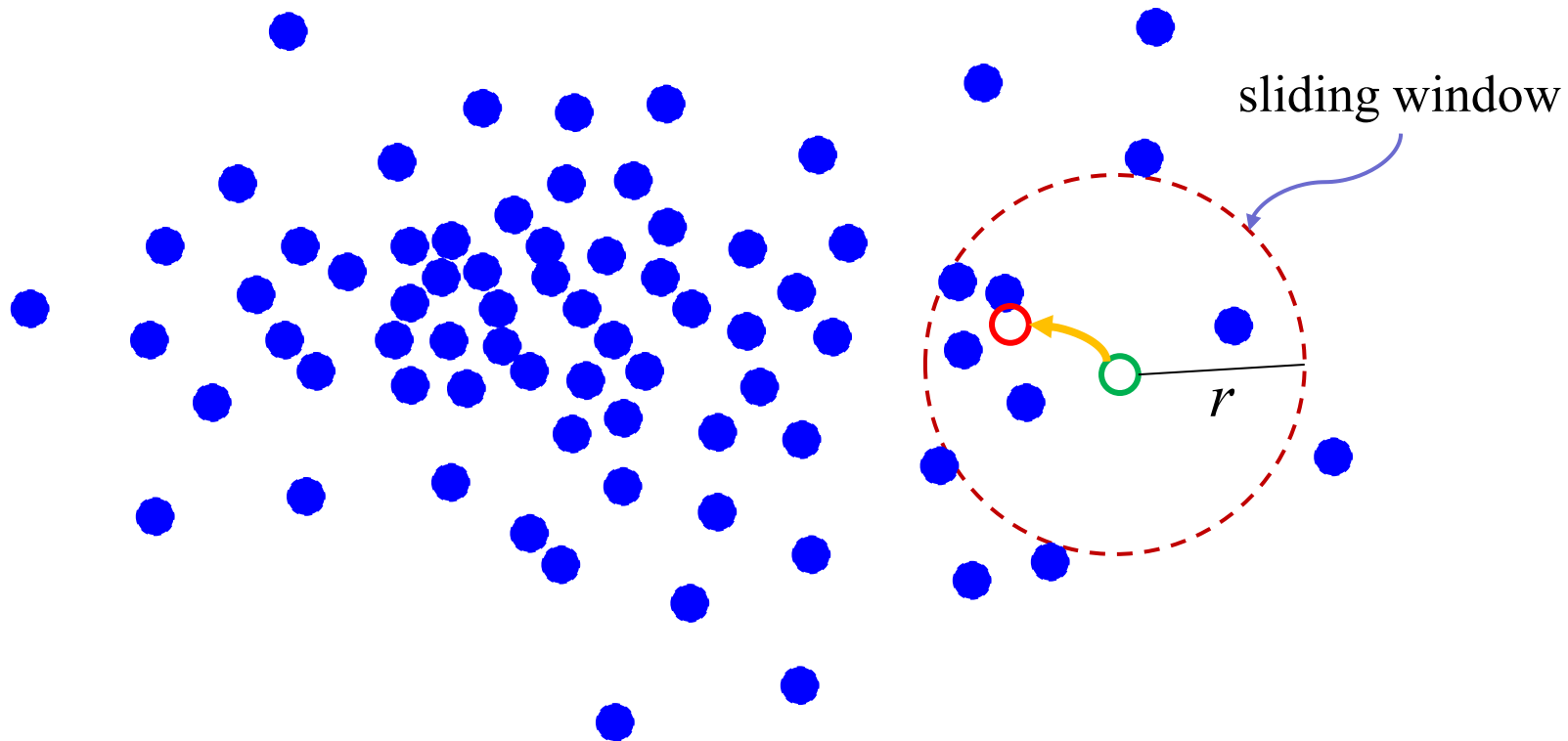
Mean-Shift Clustering

- **mean shift** clustering
 - **sliding-window**-based algorithm
 - find dense areas of data points
- **centroid**-based algorithm
 - updating candidates for center points to be the **mean** of the points within the sliding-window

$$m(\mathbf{x}) = \frac{\sum_{\mathbf{x}_i \in N(\mathbf{x})} K(\mathbf{x}_i - \mathbf{x}) \mathbf{x}_i}{\sum_{\mathbf{x}_i \in N(\mathbf{x})} K(\mathbf{x}_i - \mathbf{x})} \quad K(\mathbf{x}_i - \mathbf{x}) = \exp(-c \|\mathbf{x}_i - \mathbf{x}\|^2)$$

- candidate windows are then filtered in a post-processing stage to eliminate near-duplicates, forming the final set of center points and their corresponding groups

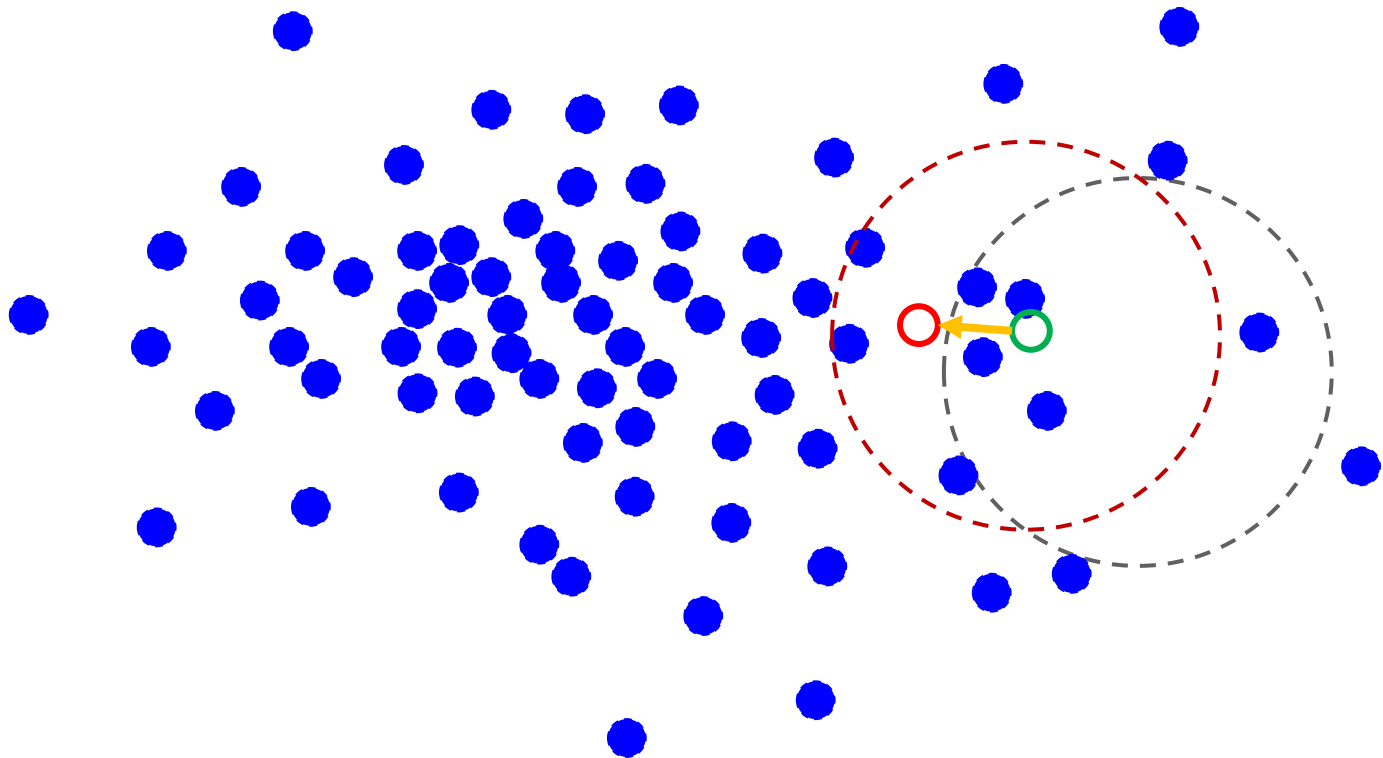
Mean-Shift with a Single Center



Center of the region

Center of the mass

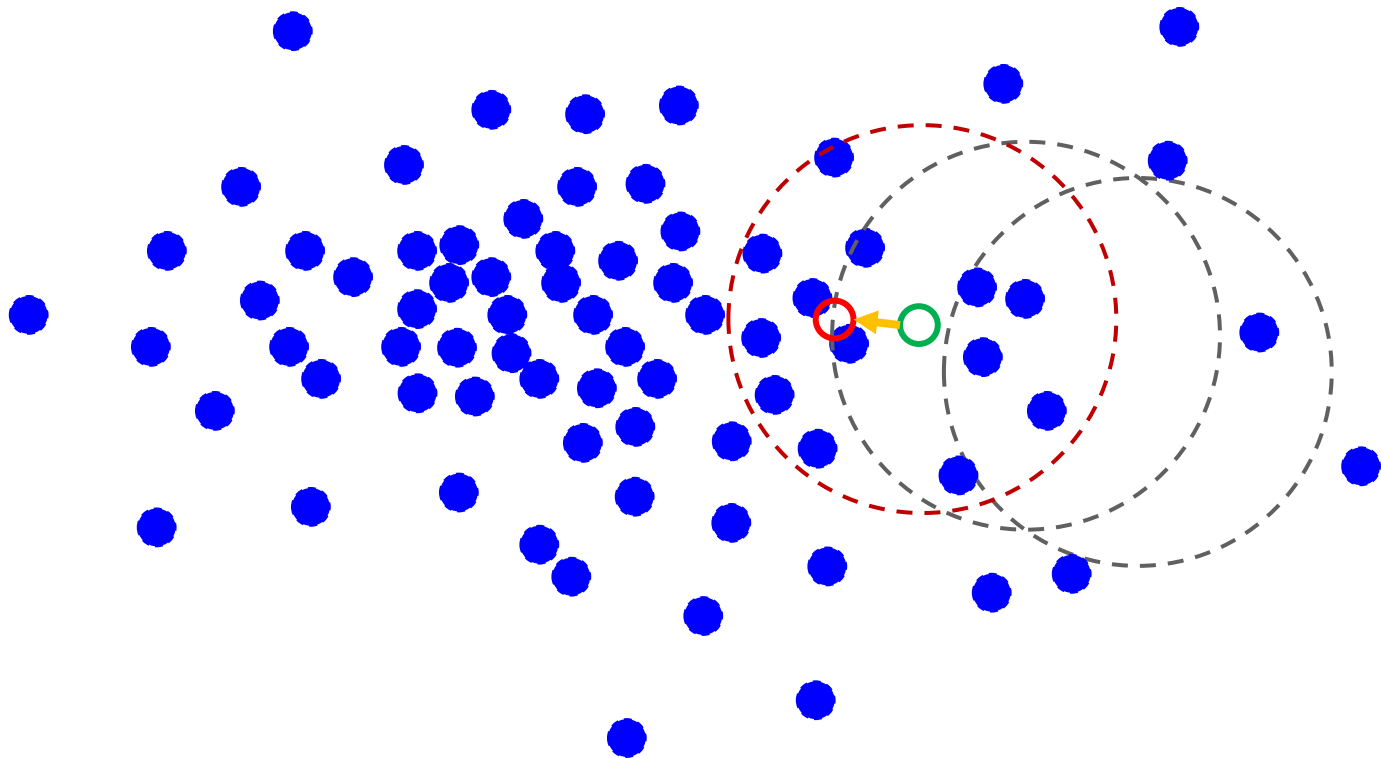
Mean-Shift with a Single Center



Center of the region

Center of the mass

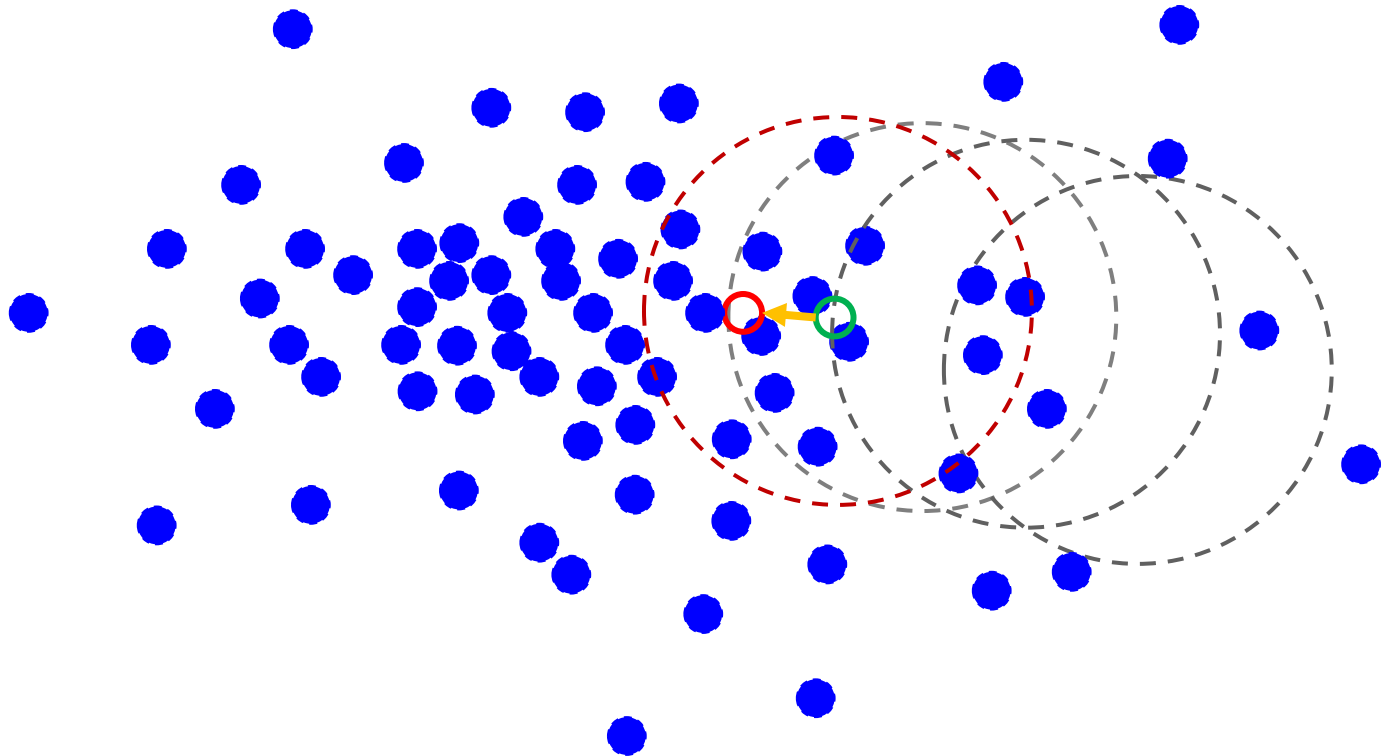
Mean-Shift with a Single Center



Center of the region

Center of the mass

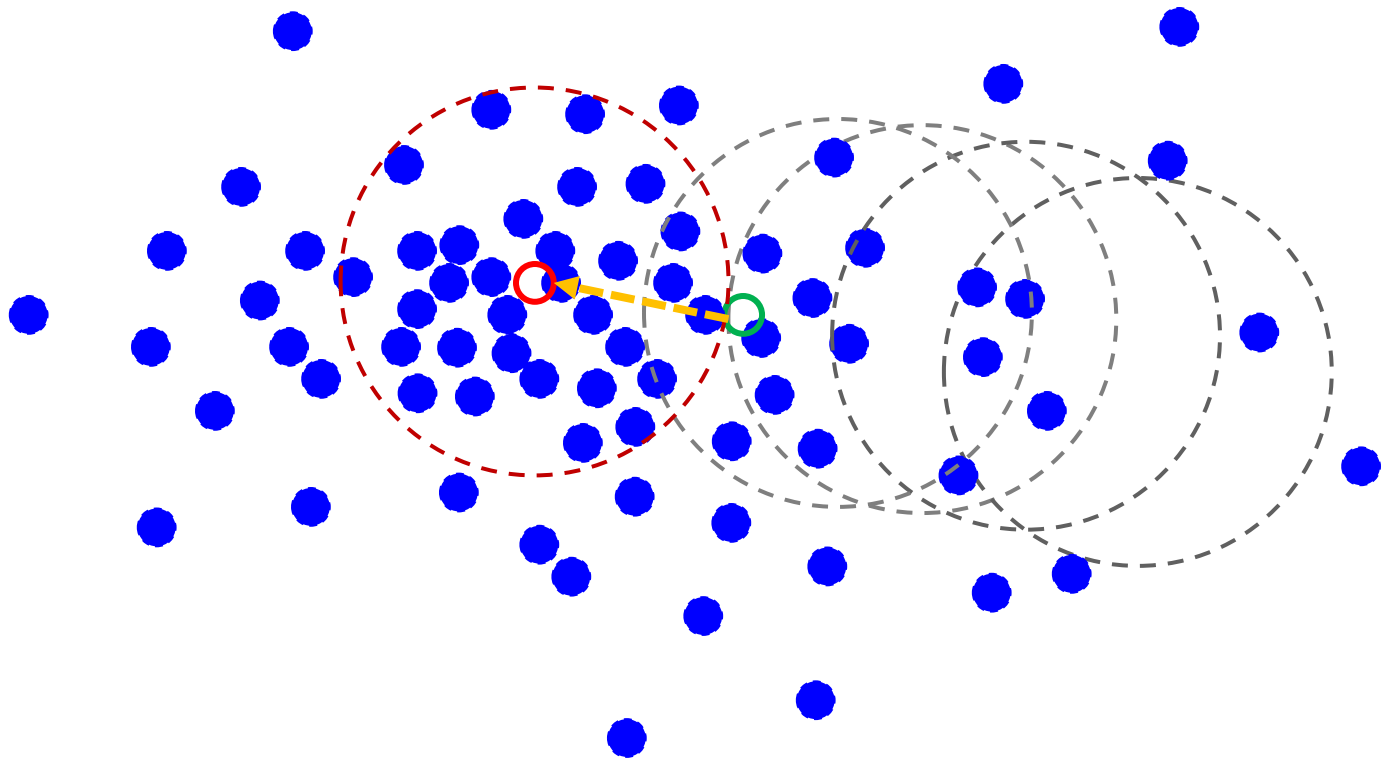
Mean-Shift with a Single Center



Center of the region

Center of the mass

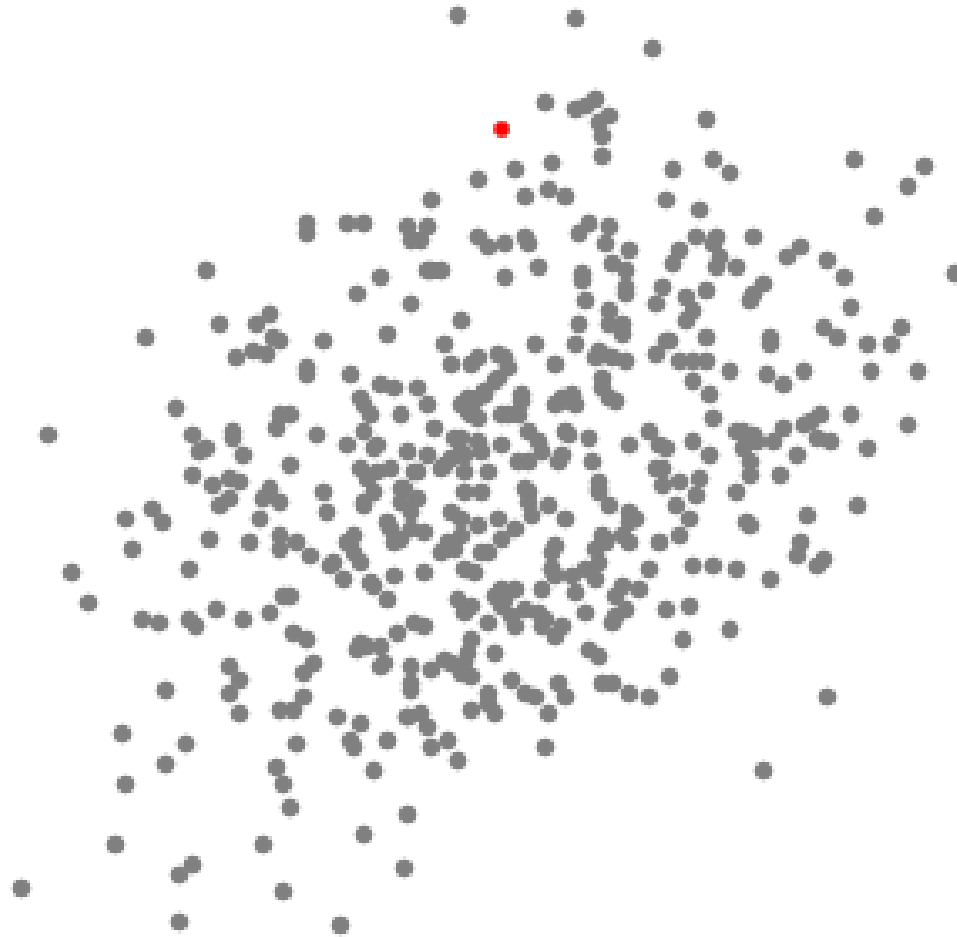
Mean-Shift with a Single Center



Center of the region

Center of the mass

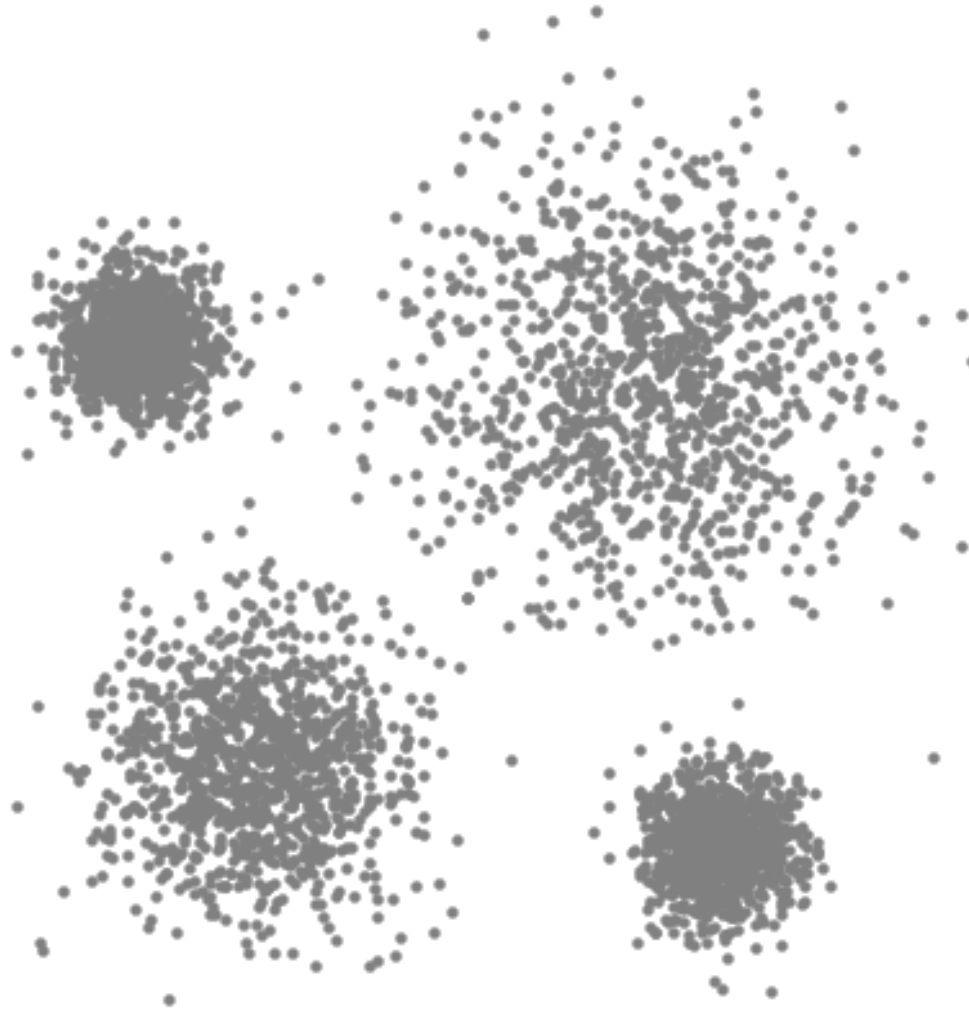
Mean-Shift with a Single Center



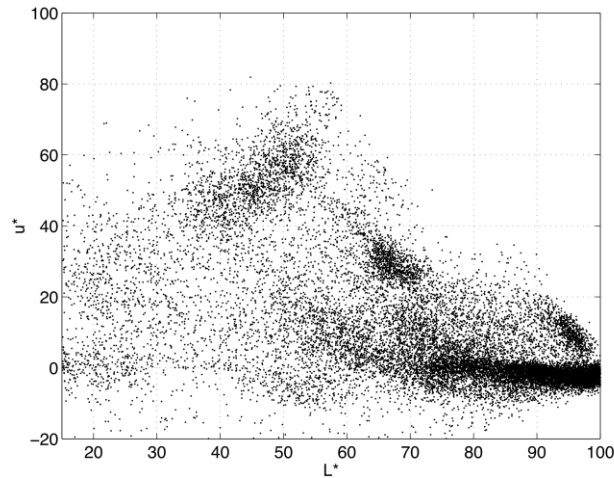
Mean-Shift Clustering

1. Begin with a **circular sliding window** centered at a point C (randomly selected) and having **radius r** as the kernel. Mean shift is a **hill-climbing algorithm** that involves shifting this kernel iteratively to a higher density region on each step until convergence.
2. At every iteration, the sliding window is shifted towards regions of higher density by **shifting the center point to the mean** of the points within the window (hence the name).
3. We continue shifting the sliding window according to the mean until there is no direction at which a shift can accommodate more points inside the kernel.
4. **This process of steps 1 to 3 is done with many sliding windows until all points lie within a window.**
5. When multiple sliding windows overlap the window containing the most points is preserved.
6. The data points are then clustered according to the sliding window in which they reside.

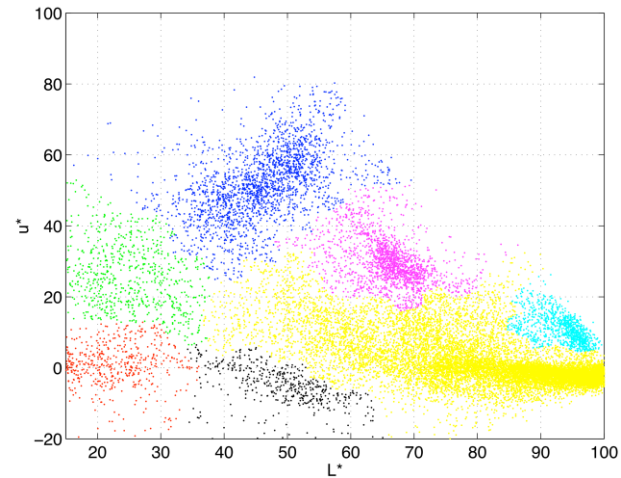
Mean-Shift with Multiple Centers



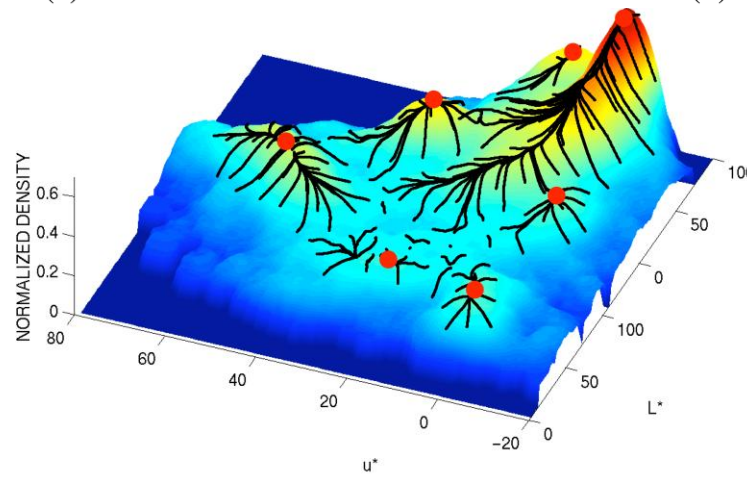
Mean-Shift with Multiple Centers



(a)



(b)



(c)

D. Comaniciu and P. Meer. Mean Shift: A Robust Approach toward Feature Space Analysis. TPAMI, 2002.

Mean-Shift Clustering

1. In contrast to K-means clustering, there is no need to select the number of clusters as mean-shift automatically discovers this. That's a massive advantage.
2. The fact that the cluster centers converge towards the points of maximum density is also quite desirable as it is quite intuitive to understand and fits well in a naturally data-driven sense.
3. There is no prior assumption on the prior shape of clusters.
4. The drawback is that the selection of the **window size/radius "r"** can be non-trivial.
5. Computationally expensive.

DBSCAN

Density-Based Spatial Clustering of Applications with Noise (DBSCAN) is a density-based clustered algorithm similar to mean-shift, but with a couple of notable advantages.

1. Begin with an arbitrary starting data point (**not visited**) x . The **neighborhood** of x : all points within the **ϵ distance** to x .
2. Sufficient number of points (**$> \text{minPoints}$**) within neighborhood: x becomes the first point in the new cluster.
3. Otherwise, x will be labeled as **noise** (later this noisy point might become the part of another cluster).
4. In both cases x is marked as **"visited"**.
5. For this first point in the new cluster, the points within its ϵ distance neighborhood also become part of the same cluster. This procedure of **making all points in the ϵ neighborhood belong to the same cluster** is then repeated for all of the new points that have been just added to the cluster group.
6. This process repeats **until all points are marked as visited**.

DBSCAN

- Input: $X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$, ϵ , minPoints

1 for each unvisited point $\mathbf{x} \in X$ do:

2 mark $\mathbf{x}$ as visited

3 $N = \text{GetNeighbors}(\mathbf{x}, \epsilon)$

4 **if** $|N| < \text{minPoints}$ **then:**

5 mark $\mathbf{x}$ as noise

6 **else**

7 $C \leftarrow \{\mathbf{x}\}$

8 for each point $\mathbf{x}' \in N$ do:

9 $N \leftarrow N \setminus \mathbf{x}'$

10 **if** $\mathbf{x}'$ is not visited **then:**

11 mark $\mathbf{x}'$ as visited

12 **if** $\mathbf{x}'$ is not a member of any cluster **then:**

13 $C \leftarrow C \cup \mathbf{x}'$

14 $N' = \text{GetNeighbors}(\mathbf{x}', \epsilon)$

15 **if** $|N'| > \text{minPoints}$ **then:**

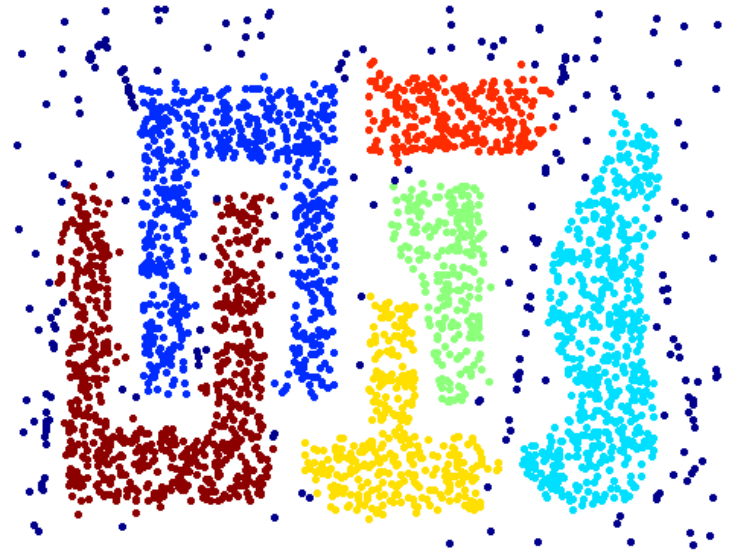
16 $N \leftarrow N \cup N'$

- ✓ If there are at least ‘minPoints’ points within a radius of ‘ ϵ ’ to the point then we consider all these points to be part of the same cluster.
- ✓ The clusters are then expanded by recursively repeating the neighborhood calculation for each neighboring point.

DBSCAN



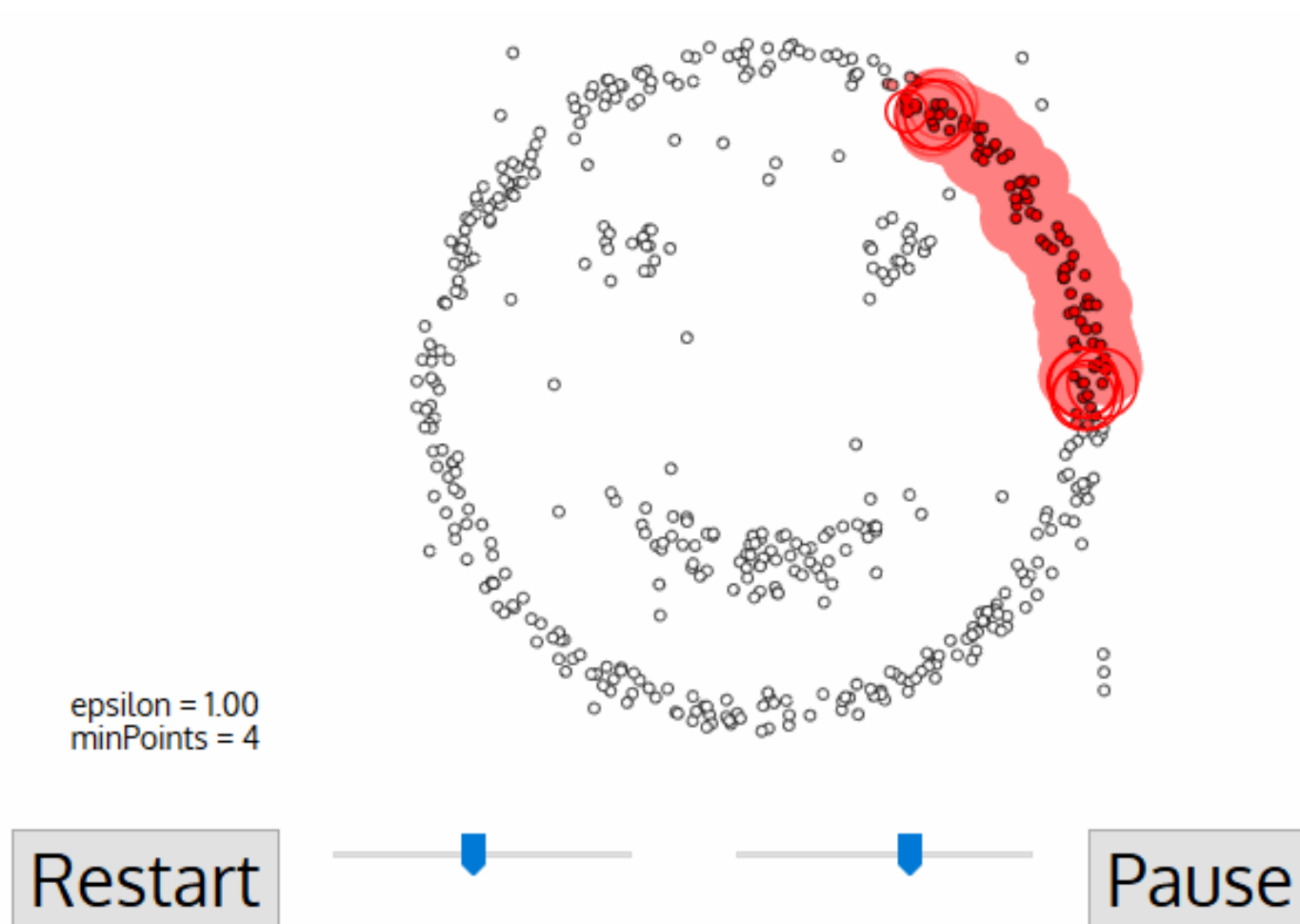
Original Points



Clusters

$\epsilon = 10$, minPoints = 4

DBSCAN



<https://www.naftaliharris.com/blog/visualizing-dbscan-clustering/>

<https://www.naftaliharris.com/blog/visualizing-k-means-clustering/>

DBSCAN

1. DBSCAN does not require a pre-set number of clusters at all.
2. It also identifies outliers as noises, unlike mean-shift which simply throws them into a cluster even if the data point is very different.
3. Additionally, it can find arbitrarily sized and arbitrarily shaped clusters quite well.
4. The main drawback of DBSCAN is that it doesn't perform as well as others when the clusters are of varying density. This is because the setting of the **distance threshold ϵ** and **minPoints** for identifying the neighborhood points will vary from cluster to cluster when the density varies.
5. This drawback also occurs with very high-dimensional data since again the distance threshold ϵ becomes challenging to estimate.

Gaussian Mixture Models

Gaussian Distribution

- Multivariate Gaussian

$$\mathcal{N}(\mathbf{x} | \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi |\boldsymbol{\Sigma}|)^{1/2}} \exp \left\{ -\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}) \right\}$$

Diagram illustrating the components of the Multivariate Gaussian distribution formula:

- $\boldsymbol{\mu}$: mean
- $\boldsymbol{\Sigma}$: covariance
- $(2\pi |\boldsymbol{\Sigma}|)^{1/2}$: Normalizer
- $(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})$: Mahalanobis distance

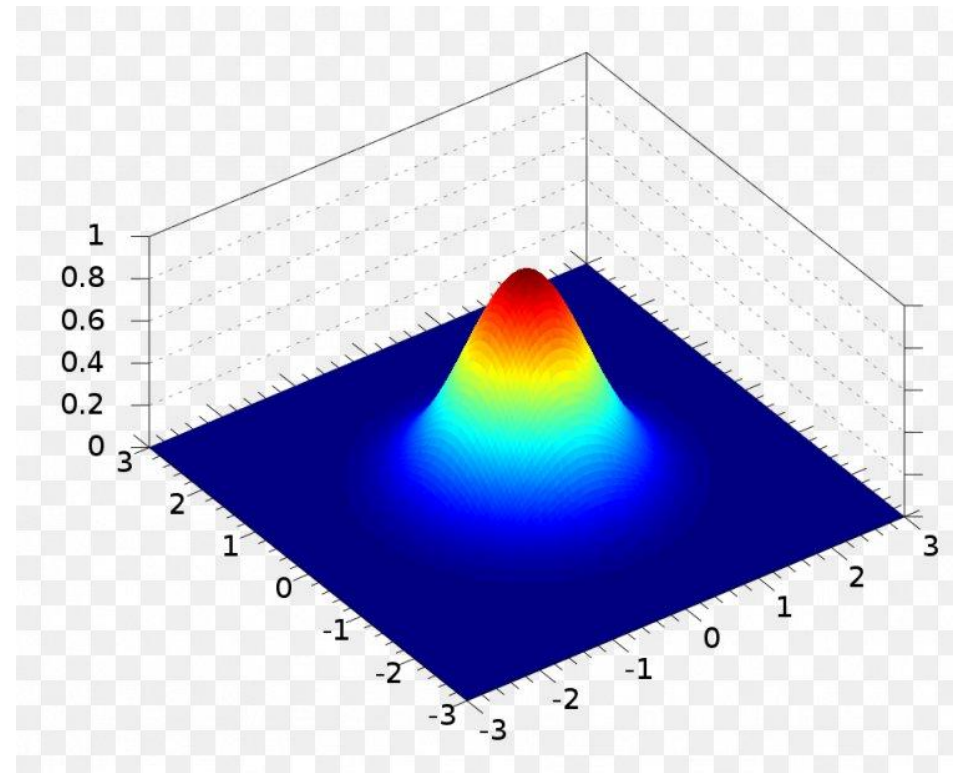
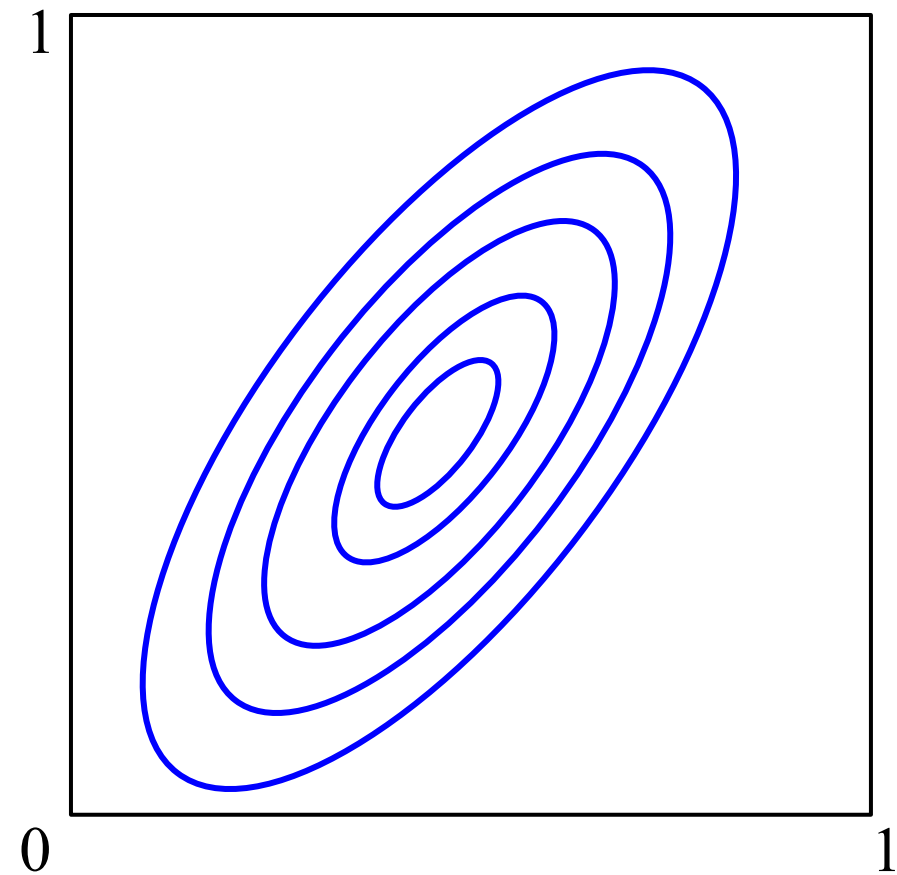
- Define precision to be the inverse of the covariance

$$\boldsymbol{\Lambda} = \boldsymbol{\Sigma}^{-1}$$

- In 1-dimension

$$\tau = \frac{1}{\sigma^2}$$

Example: 2-dim Gaussian Distribution



Likelihood Function

- Data set

$$D = \{\mathbf{x}_n\} \quad n = 1, \dots, N$$

- Assume observed data points generated independently

$$p(D|\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \prod_{n=1}^N \mathcal{N}(\mathbf{x}_n|\boldsymbol{\mu}, \boldsymbol{\Sigma})$$

- Viewed as a function of the parameters, this is known as the *likelihood function*

Maximum Likelihood

- Set the parameters by maximizing the likelihood function
- Equivalently maximize the log likelihood

$$\begin{aligned}\ln p(D|\boldsymbol{\mu}, \boldsymbol{\Sigma}) &= -\frac{N}{2} \ln |\boldsymbol{\Sigma}| - \frac{N}{2} \ln(2\pi) \\ &\quad - \frac{1}{2} \sum_{n=1}^N (\mathbf{x}_n - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x}_n - \boldsymbol{\mu})\end{aligned}$$

Maximum Likelihood Solution

- Maximizing w.r.t. the mean gives the *sample mean*

$$\mu_{\text{ML}} = \frac{1}{N} \sum_{n=1}^N \mathbf{x}_n$$

- Maximizing w.r.t. covariance gives the *sample covariance*

$$\Sigma_{\text{ML}} = \frac{1}{N} \sum_{n=1}^N (\mathbf{x}_n - \mu_{\text{ML}})(\mathbf{x}_n - \mu_{\text{ML}})^{\top}$$

Gaussian Mixtures

- Linear super-position of Gaussians

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

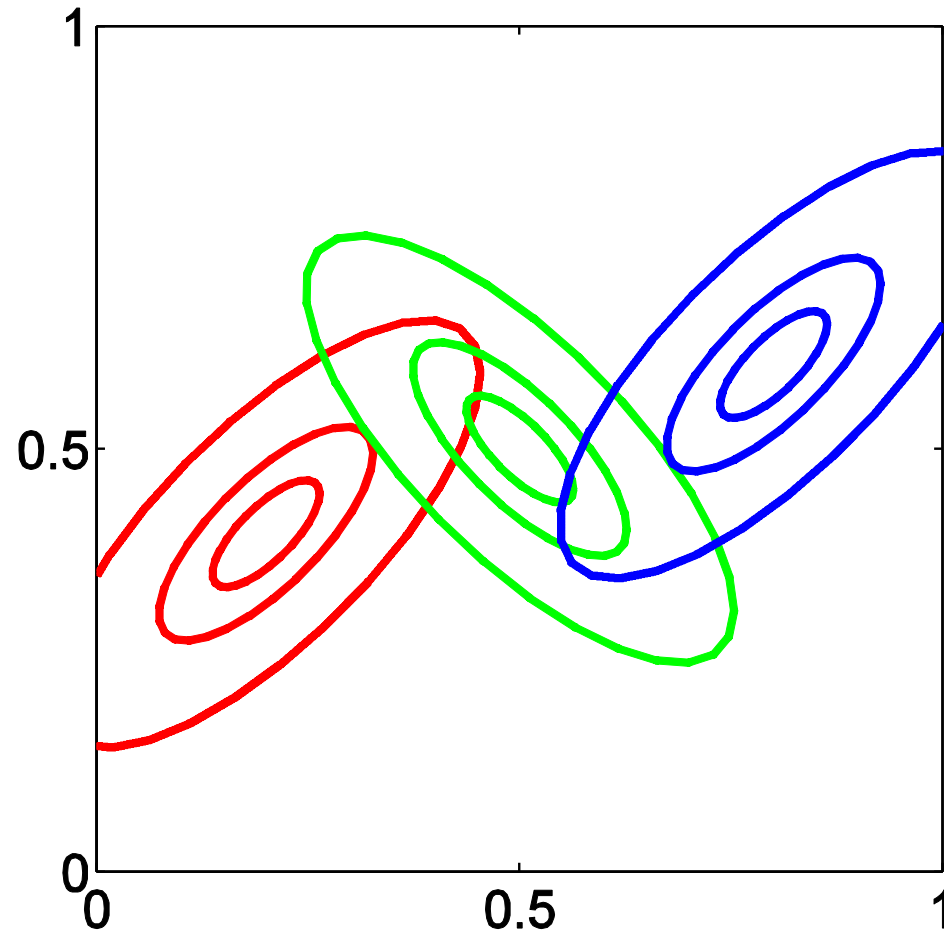
- Normalization and positivity require

$$\sum_{k=1}^K \pi_k = 1 \quad 0 \leq \pi_k \leq 1$$

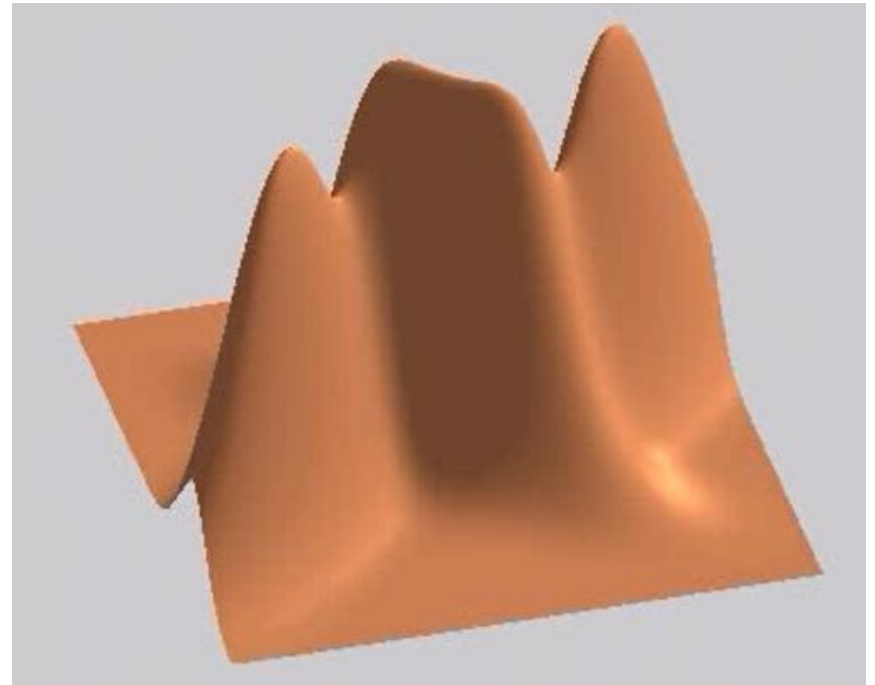
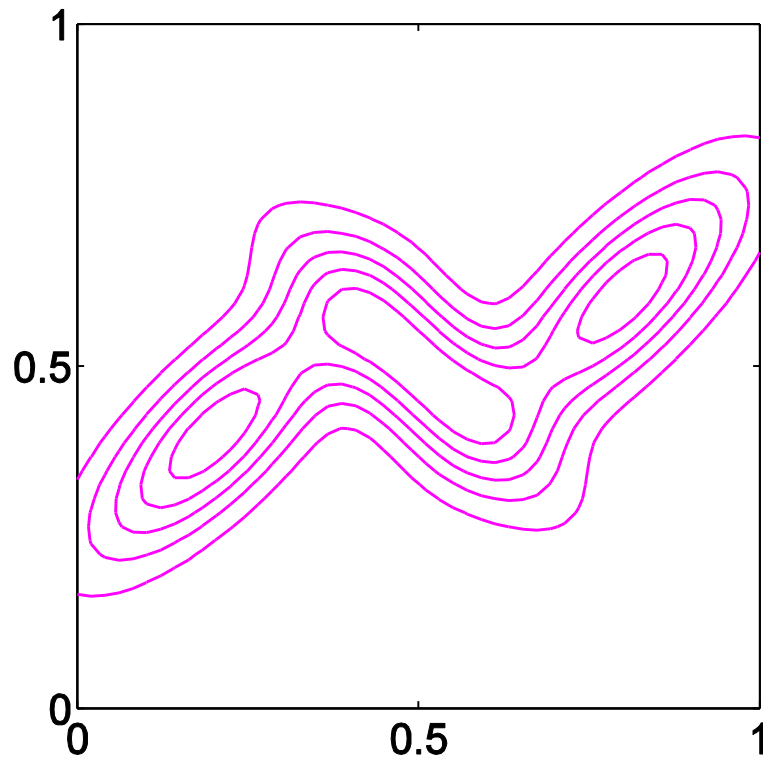
- Can interpret the mixing coefficients as prior probabilities

$$p(\mathbf{x}) = \sum_{k=1}^K p(k) p(\mathbf{x} | k)$$

Example: Mixture of 3 Gaussians



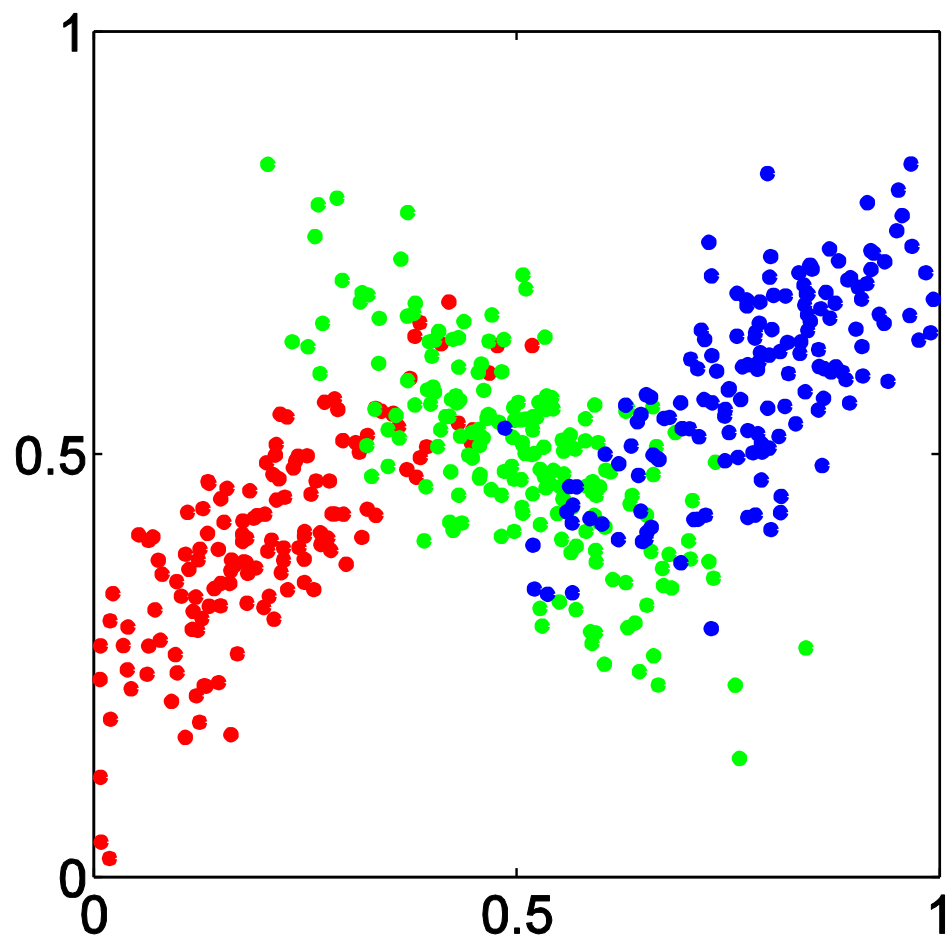
Contours of Probability Distribution



Sampling from Gaussian Mixtures

- To generate a data point:
 - first pick one of the components with probability π_k
 - then draw a sample $\mathbf{x}_n$ from that component
- Repeat these two steps for each new data point

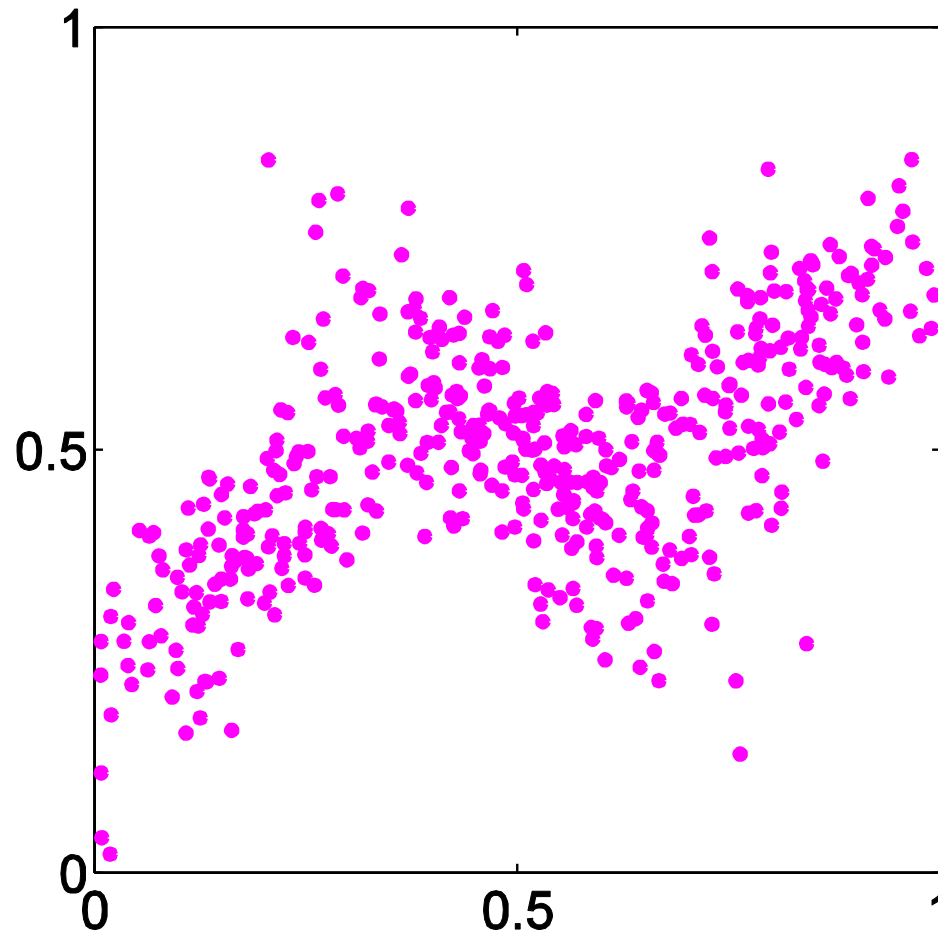
Synthetic Data Set



Fitting the Gaussian Mixture

- We wish to invert this process – given the data set, find the corresponding parameters:
 - mixing coefficients
 - means
 - covariances
- If we knew which component generated each data point, the maximum likelihood solution would involve fitting each component to the corresponding cluster
- Problem: the data set is unlabelled
- We shall refer to the labels as *latent* (= hidden) variables

Synthetic Data Set Without Labels

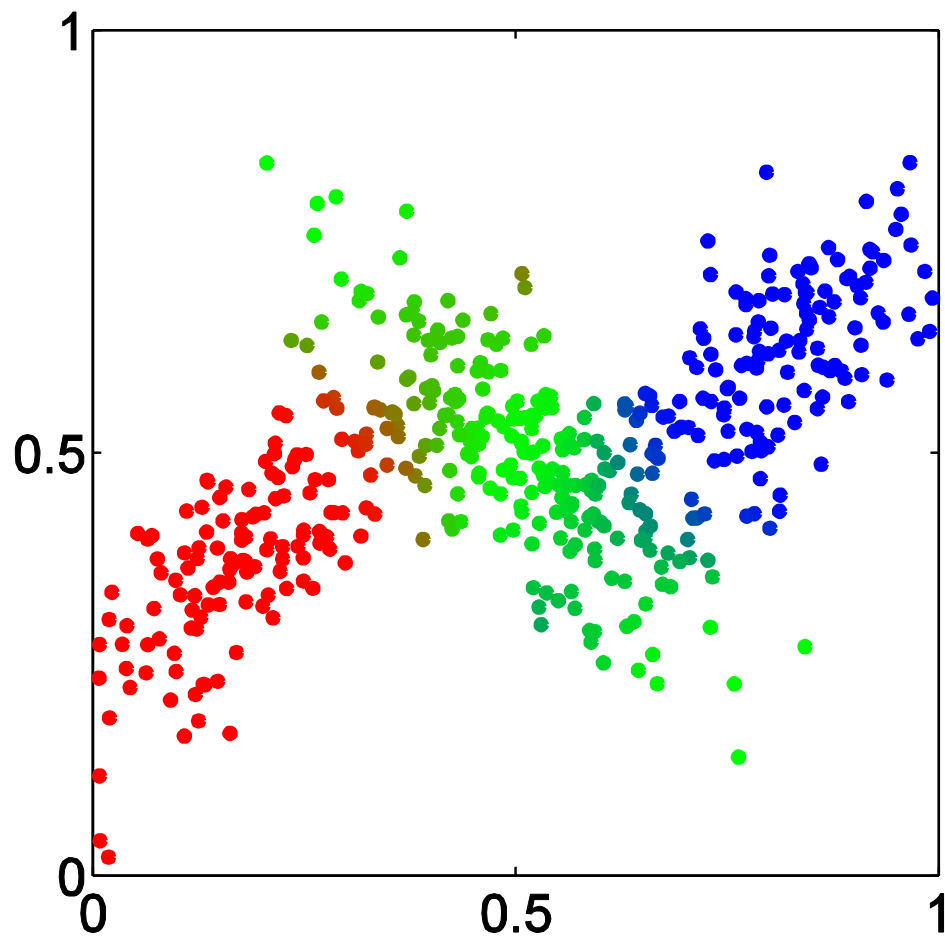


Posterior Probabilities

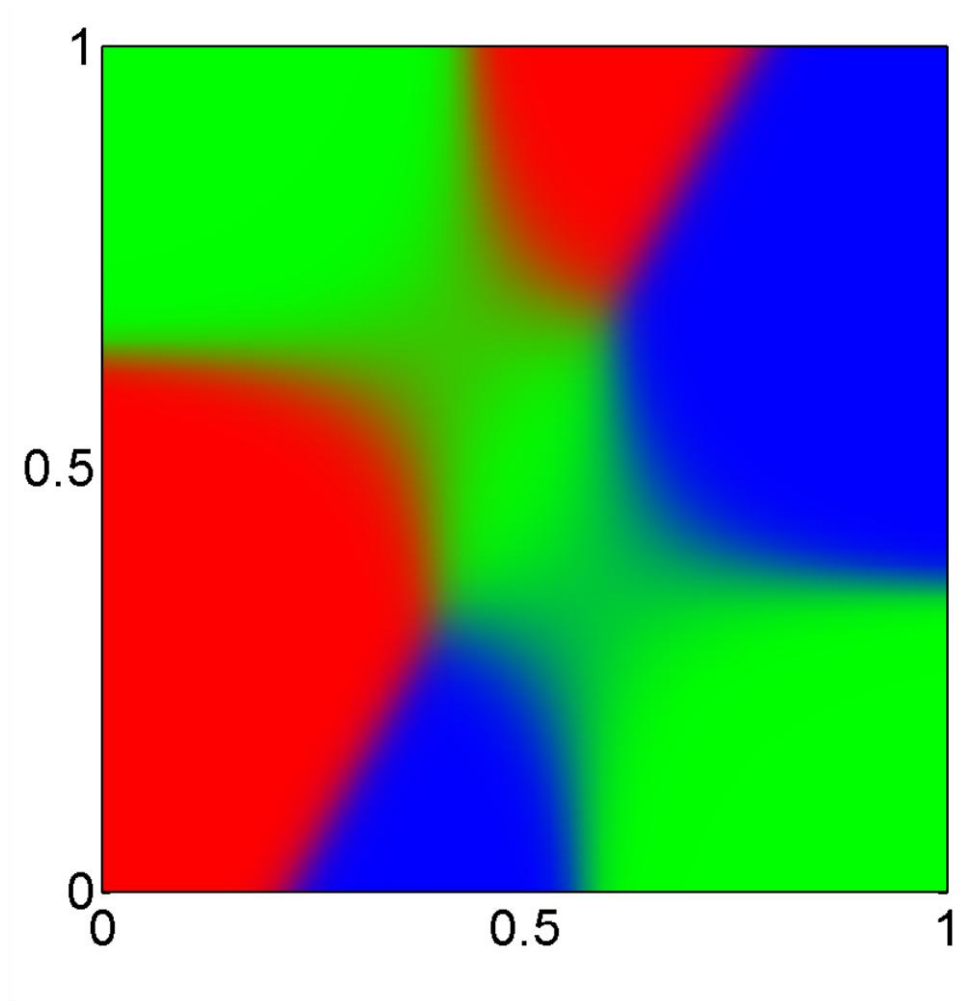
- We can think of the mixing coefficients as prior probabilities for the components
- For a given value of $\mathbf{x}$ we can evaluate the corresponding posterior probabilities, called *responsibilities*
- These are given from Bayes' theorem by

$$\begin{aligned}\gamma_k(\mathbf{x}) \equiv p(k|\mathbf{x}) &= \frac{p(k)p(\mathbf{x}|k)}{p(\mathbf{x})} \\ &= \frac{\pi_k \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}\end{aligned}$$

Posterior Probabilities (colour coded)



Posterior Probability Map



Maximum Likelihood for the GMM

- The log likelihood function takes the form

$$\ln p(D|\boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \sum_{n=1}^N \ln \left\{ \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right\}$$

- Note: sum over components appears *inside* the log
- There is no closed form solution for maximum likelihood

Maximum Likelihood for the GMM

- The log likelihood function takes the form

$$\ln p(D|\boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \sum_{n=1}^N \ln \left\{ \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right\}$$

- Note: sum over components appears *inside* the log
- There is no closed form solution for maximum likelihood
- Solved by expectation-maximization (EM) algorithm or gradient ascent algorithm.

EM Algorithm – Informal Derivation

- Let us proceed by simply differentiating the log likelihood
- Setting derivative with respect to μ_j equal to zero gives

$$-\sum_{n=1}^N \frac{\pi_j \mathcal{N}(\mathbf{x}_n | \mu_j, \Sigma_j)}{\underbrace{\sum_k \pi_k \mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k)}_{\gamma_j(\mathbf{x}_n)}} \Sigma_j^{-1} (\mathbf{x}_n - \mu_j) = 0$$

giving

$$\mu_j = \frac{\sum_{n=1}^N \gamma_j(\mathbf{x}_n) \mathbf{x}_n}{\sum_{n=1}^N \gamma_j(\mathbf{x}_n)}$$

which is simply the weighted mean of the data

EM Algorithm – Informal Derivation

- Similarly for the covariances

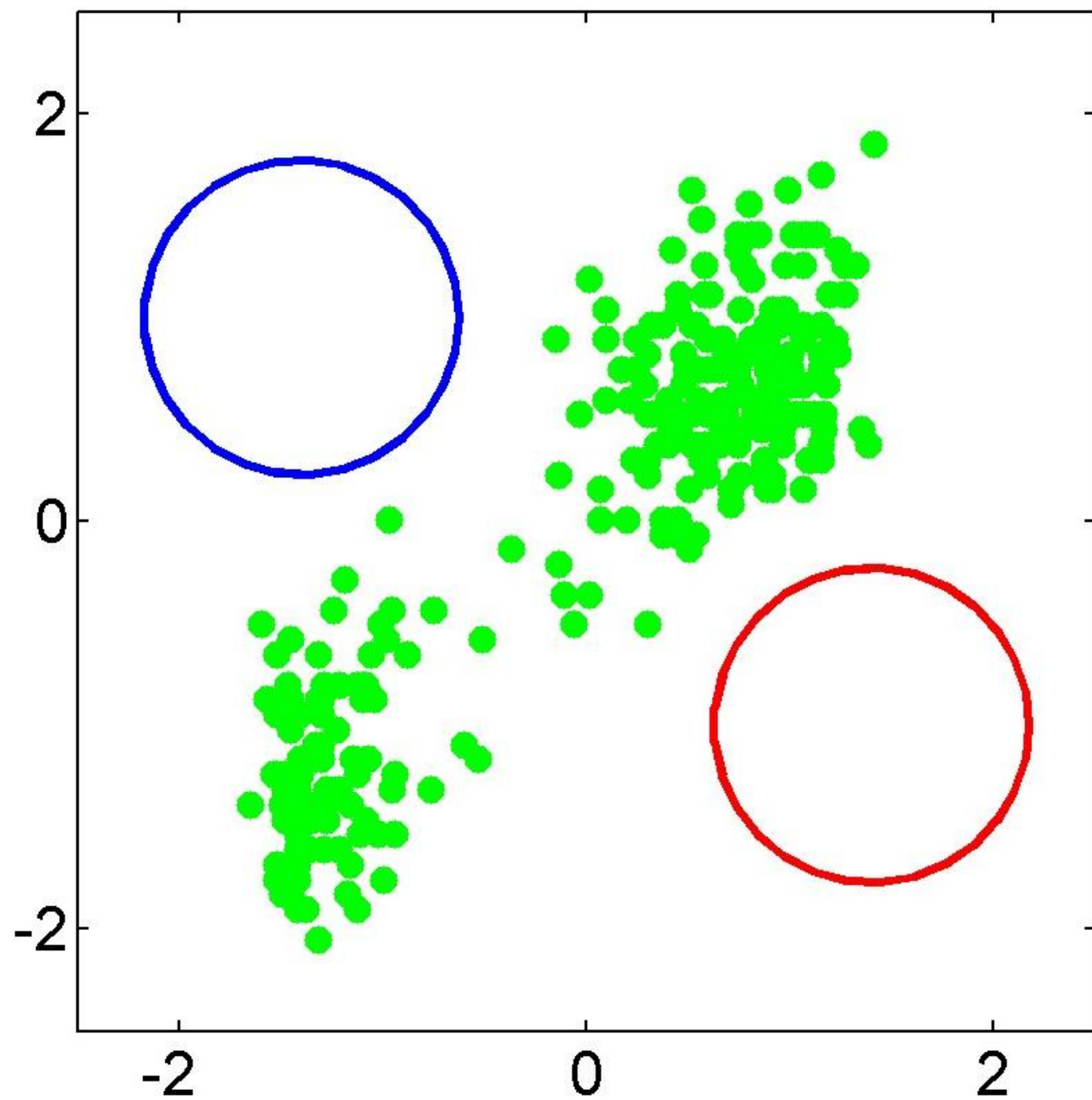
$$\Sigma_j = \frac{\sum_{n=1}^N \gamma_j(\mathbf{x}_n) (\mathbf{x}_n - \boldsymbol{\mu}_j) (\mathbf{x}_n - \boldsymbol{\mu}_j)^\top}{\sum_{n=1}^N \gamma_j(\mathbf{x}_n)}$$

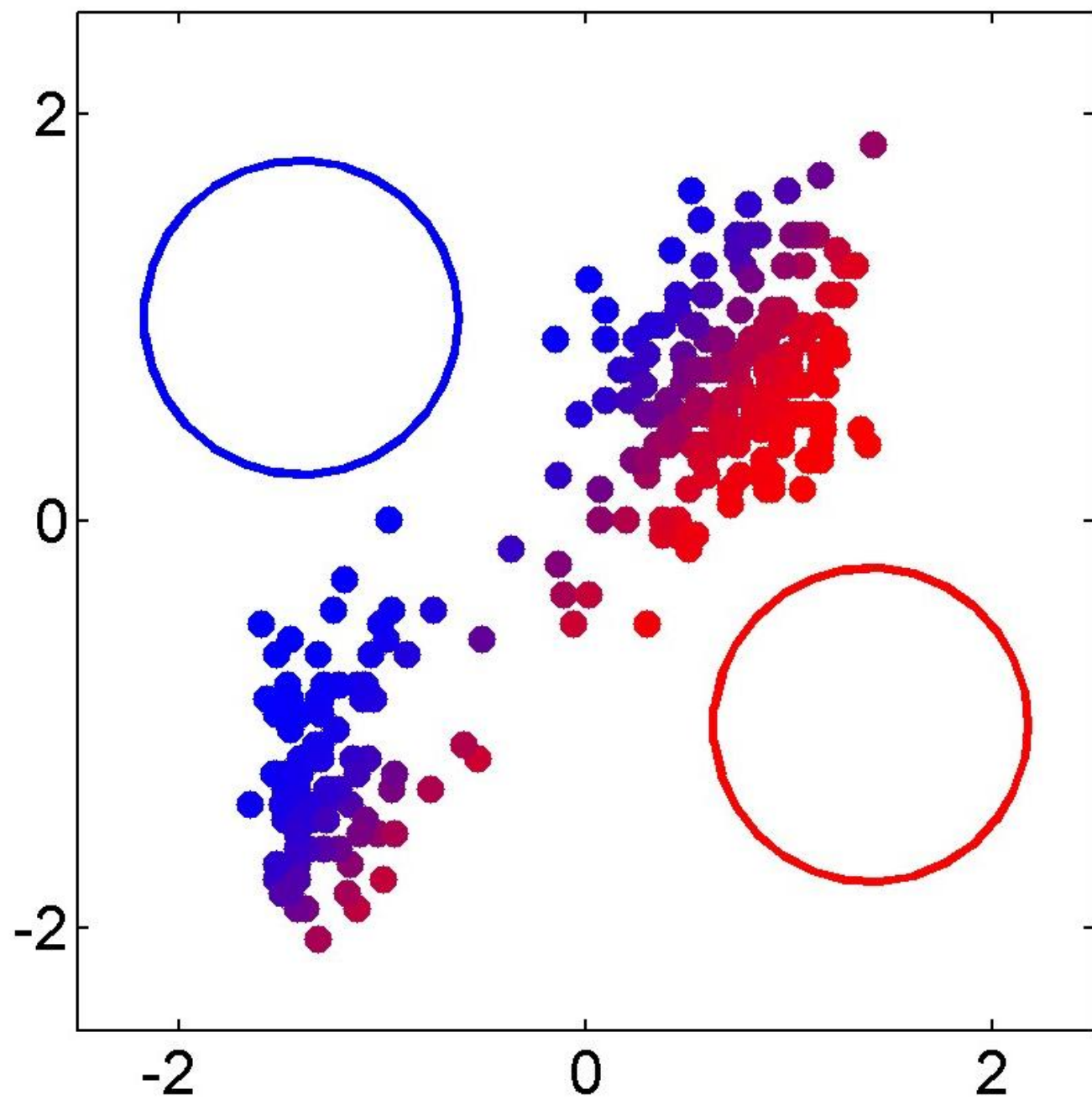
- For mixing coefficients use a Lagrange multiplier to give

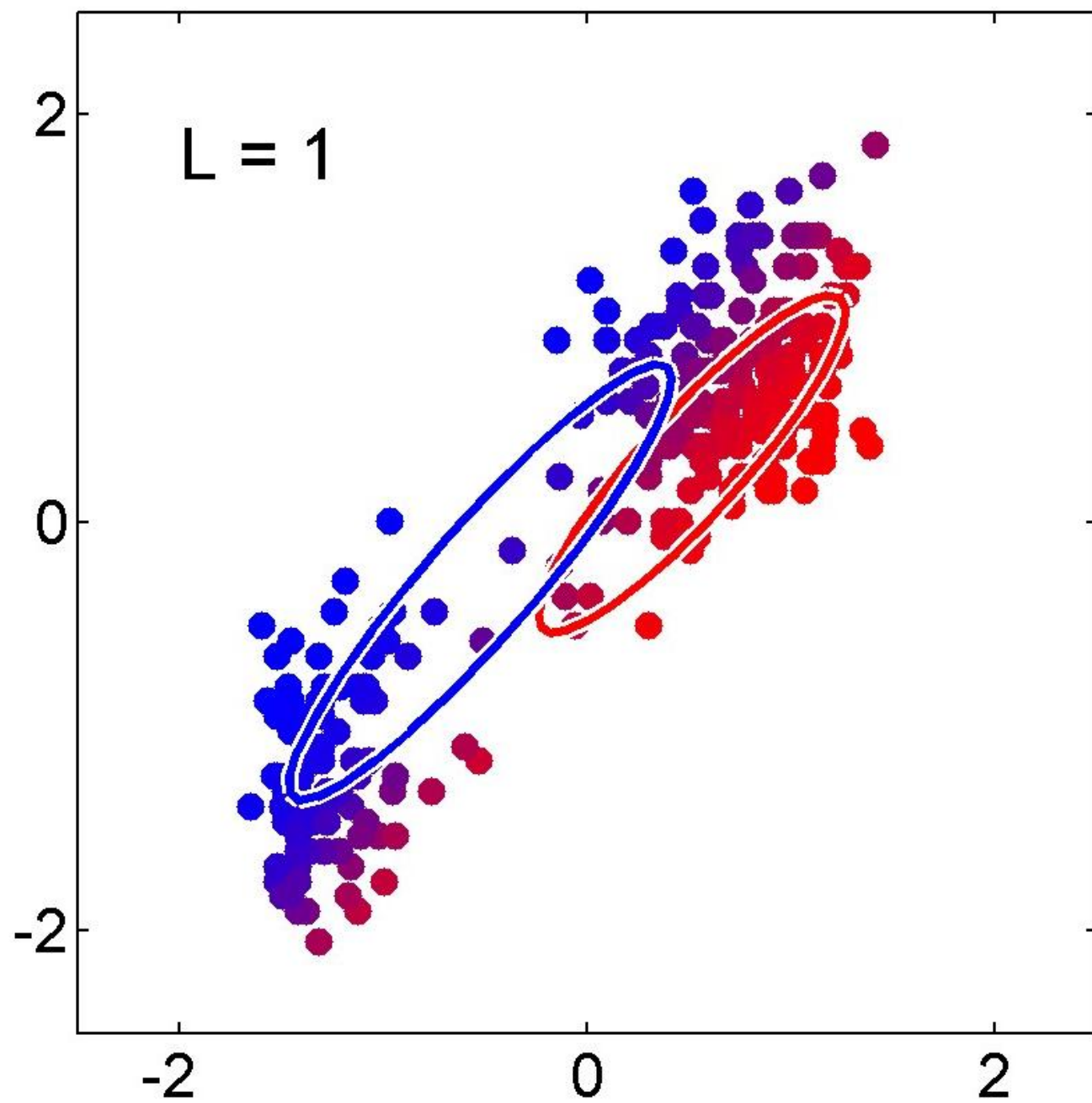
$$\pi_j = \frac{1}{N} \sum_{n=1}^N \gamma_j(\mathbf{x}_n)$$

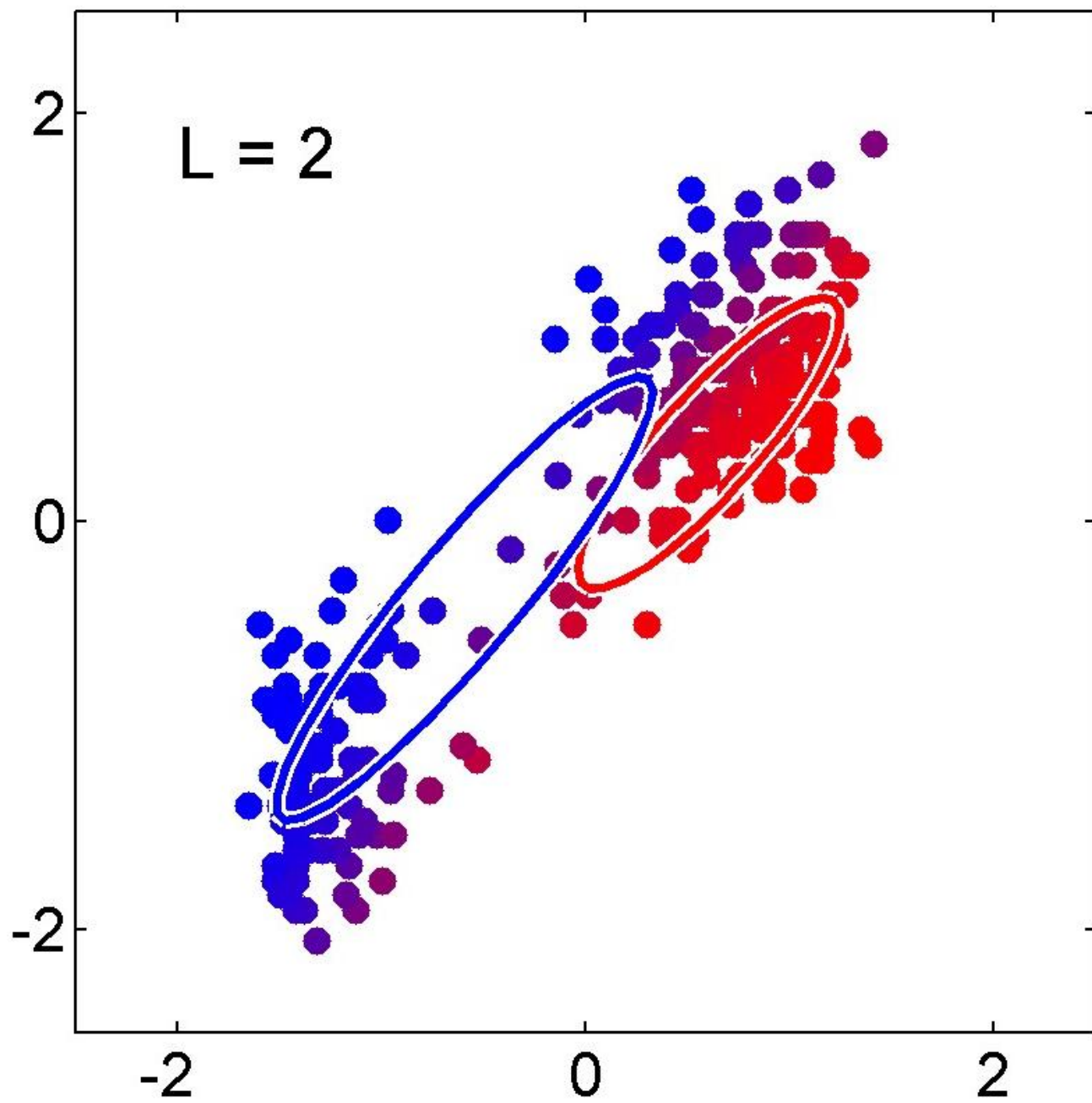
EM Algorithm – Informal Derivation

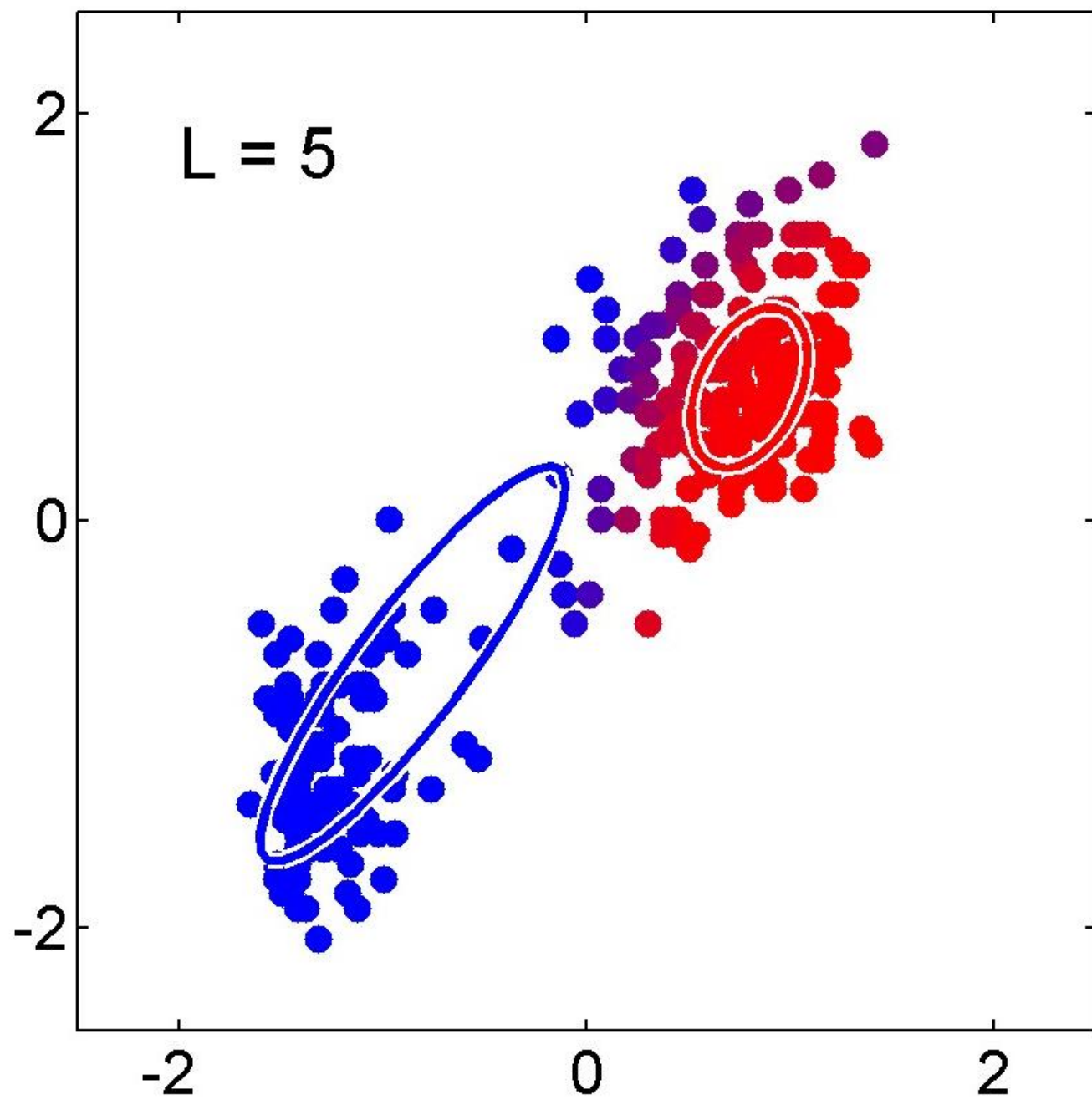
- The solutions are not closed form since they are coupled
- Suggests an iterative scheme for solving them:
 - Make initial guesses for the parameters
 - Alternate between the following two stages:
 1. E-step: evaluate responsibilities
 2. M-step: update parameters using ML results

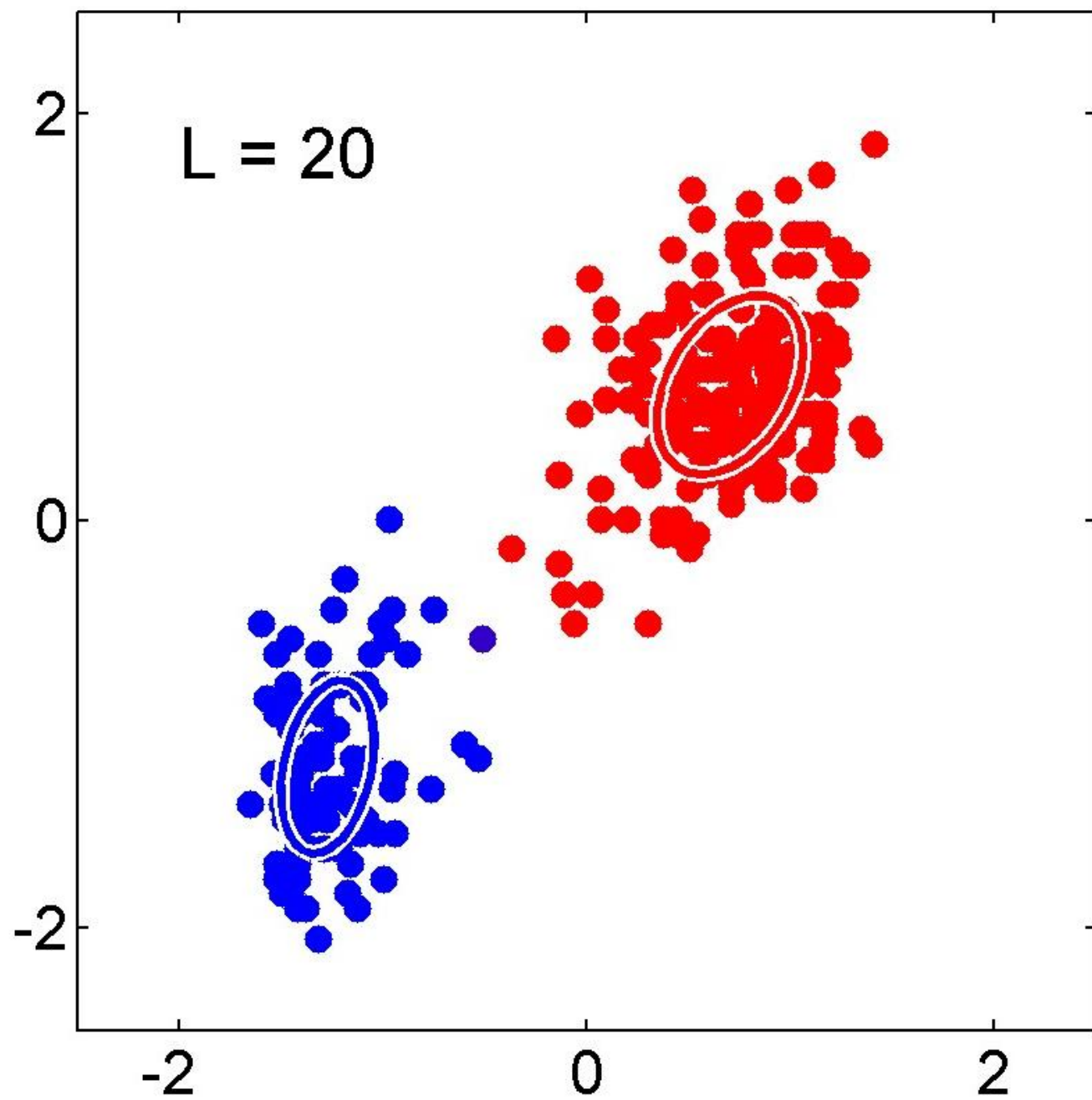












GMM Algorithm Summary

K mixture components and N samples

E-step

$$\gamma_k(\mathbf{x}) = \frac{\pi_k \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$$

M-step

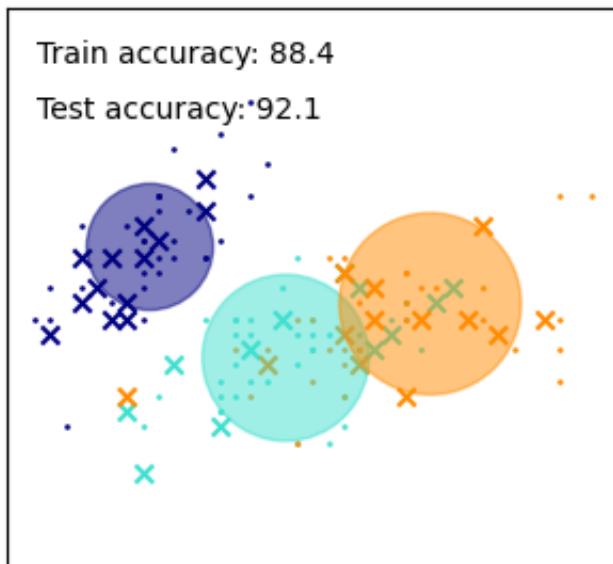
$$\boldsymbol{\mu}_j = \frac{\sum_{n=1}^N \gamma_j(\mathbf{x}_n) \mathbf{x}_n}{\sum_{n=1}^N \gamma_j(\mathbf{x}_n)} \quad \boldsymbol{\Sigma}_j = \frac{\sum_{n=1}^N \gamma_j(\mathbf{x}_n) (\mathbf{x}_n - \boldsymbol{\mu}_j)(\mathbf{x}_n - \boldsymbol{\mu}_j)^\top}{\sum_{n=1}^N \gamma_j(\mathbf{x}_n)}$$

$$\pi_j = \frac{1}{N} \sum_{n=1}^N \gamma_j(\mathbf{x}_n)$$

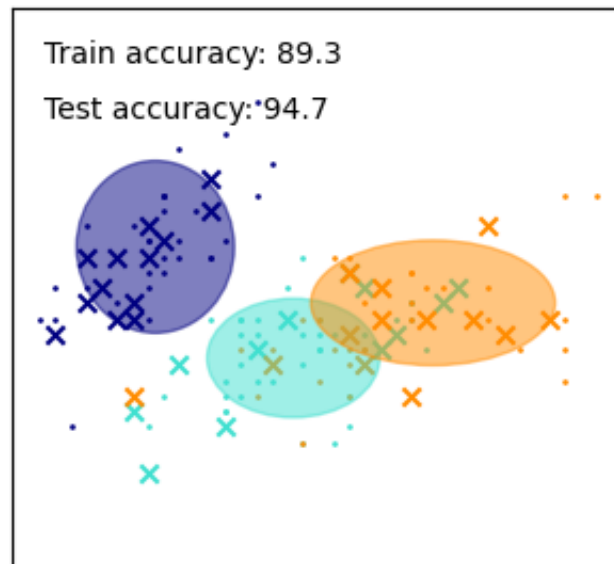
GMM Problems

- How to select K ?
- EM converge to local optimum
 - Initialization is important
- Regularization of covariance matrix
 - identity matrix
 - diagonal matrix
 - shared among different components

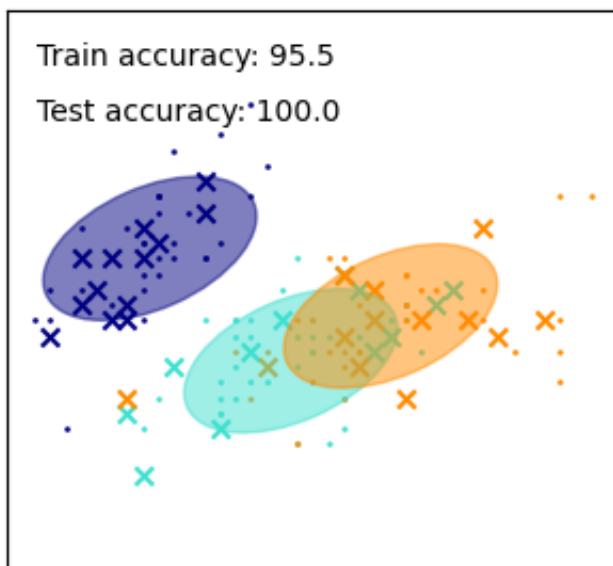
spherical



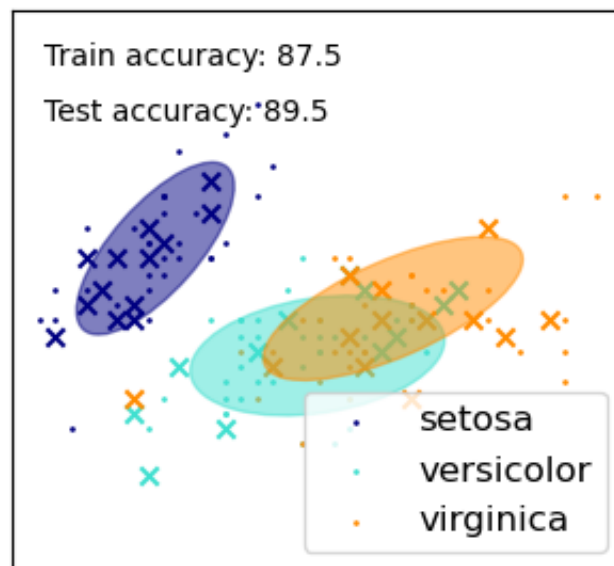
diag



tied



full



Applications of GMM

- As a soft extension of K-means
- Density estimation in Bayes classification
- GMM + HMM in speech recognition
- Data generalization using GMM
- As a general assumption for any complex distributions
- Many other applications in ML, CV, NLP

Using GMM for Generation

IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE, VOL. 40, NO. 4, APRIL 2018

849

Drawing and Recognizing Chinese Characters with Recurrent Neural Network

Xu-Yao Zhang^{ID}, Fei Yin, Yan-Ming Zhang, Cheng-Lin Liu, *Fellow, IEEE*, and Yoshua Bengio

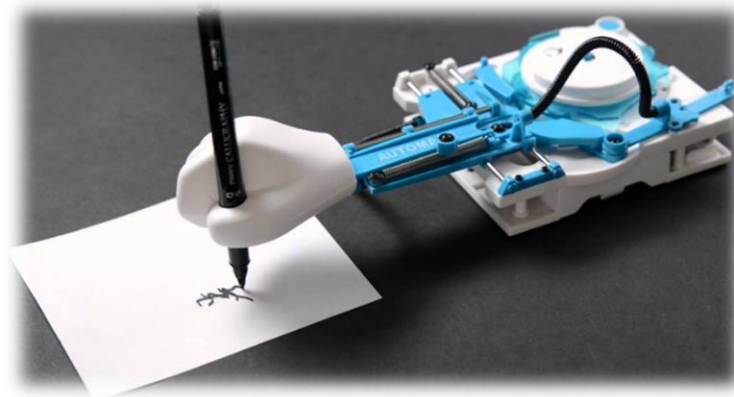
Abstract—Recent deep learning based approaches have achieved great success on handwriting recognition. Chinese characters are among the most widely adopted writing systems in the world. Previous research has mainly focused on recognizing handwritten Chinese characters. However, recognition is only one aspect for understanding a language, another challenging and interesting task is to teach a machine to automatically write (pictographic) Chinese characters. In this paper, we propose a framework by using the recurrent neural network (RNN) as both a discriminative model for recognizing Chinese characters and a generative model for drawing (generating) Chinese characters. To recognize Chinese characters, previous methods usually adopt the convolutional neural network (CNN) models which require transforming the online handwriting trajectory into image-like representations. Instead, our RNN based approach is an end-to-end system which directly deals with the sequential structure and does not require any domain-specific knowledge. With the RNN system (combining an LSTM and GRU), state-of-the-art performance can be achieved on the ICDAR-2013 competition database. Furthermore, under the RNN framework, a conditional generative model with character embedding is proposed for automatically drawing recognizable Chinese characters. The generated characters (in vector format) are human-readable and also can be recognized by the discriminative RNN model with high accuracy. Experimental results verify the effectiveness of using RNNs as both generative and discriminative models for the tasks of drawing and recognizing Chinese characters.

Index Terms—Recurrent neural network, LSTM, GRU, discriminative model, generative model, handwriting



Discriminative vs Generative

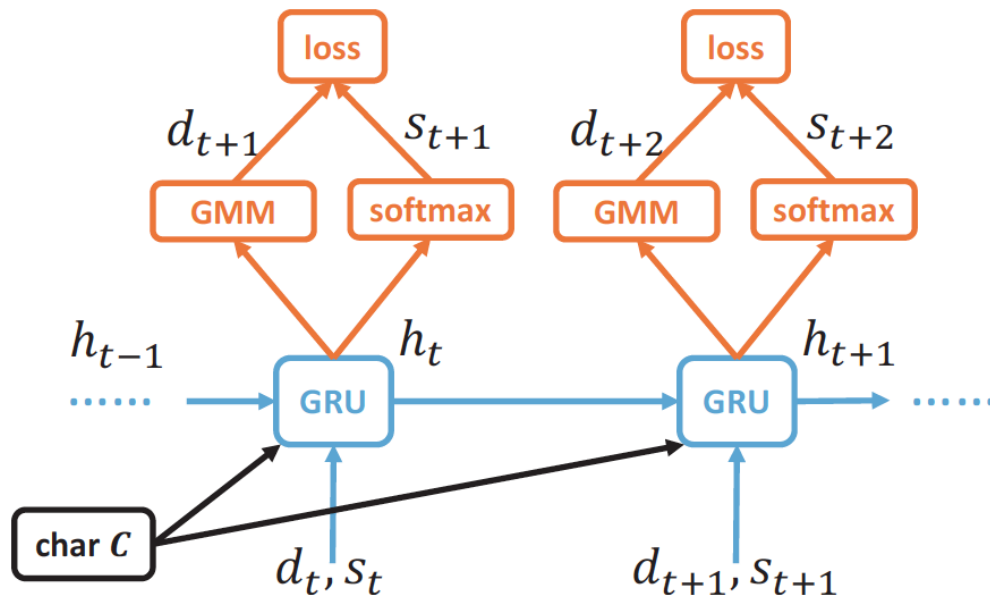
- Reading: discriminative model
- Writing: generative model
- Writing is more difficult than reading
- Use RNN as both:
 - discriminative model for online HCCR
 - generative model for drawing characters



Xu-Yao Zhang et al, Drawing and Recognizing Chinese Characters with Recurrent Neural Network, IEEE Trans. PAMI, 2018.

Generative RNN model: training process

- Condition on character embedding **c**
- Softmax for pen-state **s**
 - pen-down
 - pen-up
 - end-of-char
- GMM for pen-direction **d**
- Weighted loss function:
 - GMM: likelihood **d**
 - Softmax: cross-entropy **s**



$$\text{loss} = - \sum_t \left\{ \log (P_d(d_{t+1})) + \sum_{i=1}^3 w^i s_{t+1}^i \log(p_{t+1}^i) \right\},$$

GMM for Pen-direction

$$\left(\{\hat{\pi}^j, \hat{\mu}_x^j, \hat{\mu}_y^j, \hat{\delta}_x^j, \hat{\delta}_y^j\}_{j=1}^M\right) \in \mathbb{R}^{5M} = W_{\text{gmm}} \times o_t + b_{\text{gmm}},$$

$$\pi^j = \frac{\exp(\hat{\pi}^j)}{\sum_{j'} \exp(\hat{\pi}^{j'})} \Rightarrow \pi^j \in (0, 1), \sum_j \pi^j = 1,$$

$$\mu_x^j = \hat{\mu}_x^j \Rightarrow \mu_x^j \in \mathbb{R},$$

$$\mu_y^j = \hat{\mu}_y^j \Rightarrow \mu_y^j \in \mathbb{R},$$

$$\delta_x^j = \exp(\hat{\delta}_x^j) \Rightarrow \delta_x^j > 0,$$

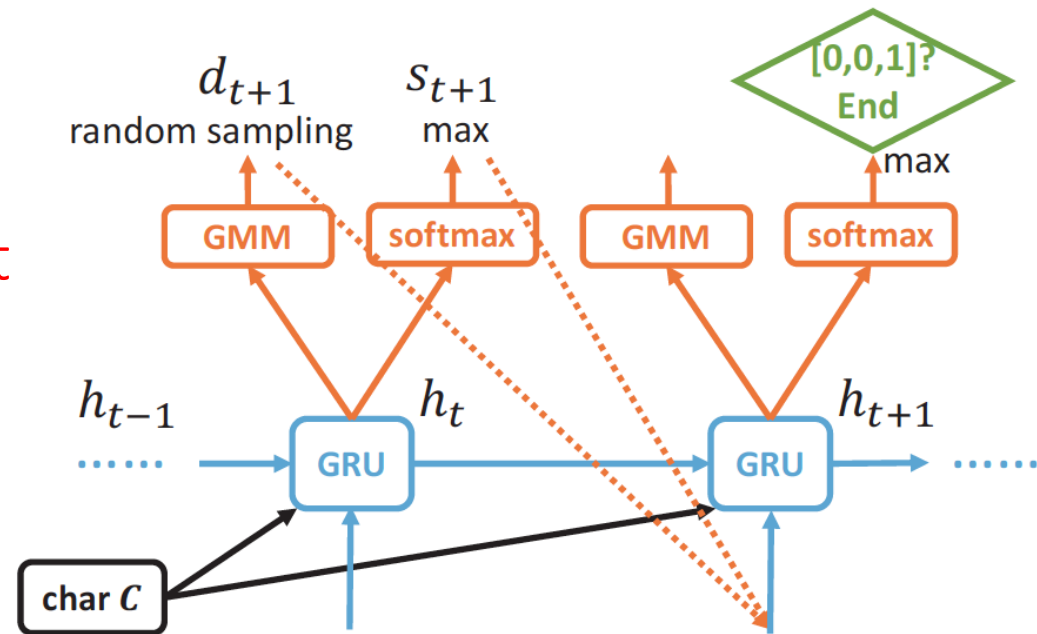
$$\delta_y^j = \exp(\hat{\delta}_y^j) \Rightarrow \delta_y^j > 0.$$

$$\mathcal{N}(x|\mu, \delta) = \frac{1}{\delta\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\delta^2}\right).$$

$$\begin{aligned} P_d(d_{t+1}) &= \sum_{j=1}^M \pi^j \mathcal{N}(d_{t+1}|\mu_x^j, \mu_y^j, \delta_x^j, \delta_y^j) \\ &= \sum_{j=1}^M \pi^j \mathcal{N}(\Delta x_{t+1}|\mu_x^j, \delta_x^j) \mathcal{N}(\Delta y_{t+1}|\mu_y^j, \delta_y^j), \end{aligned}$$

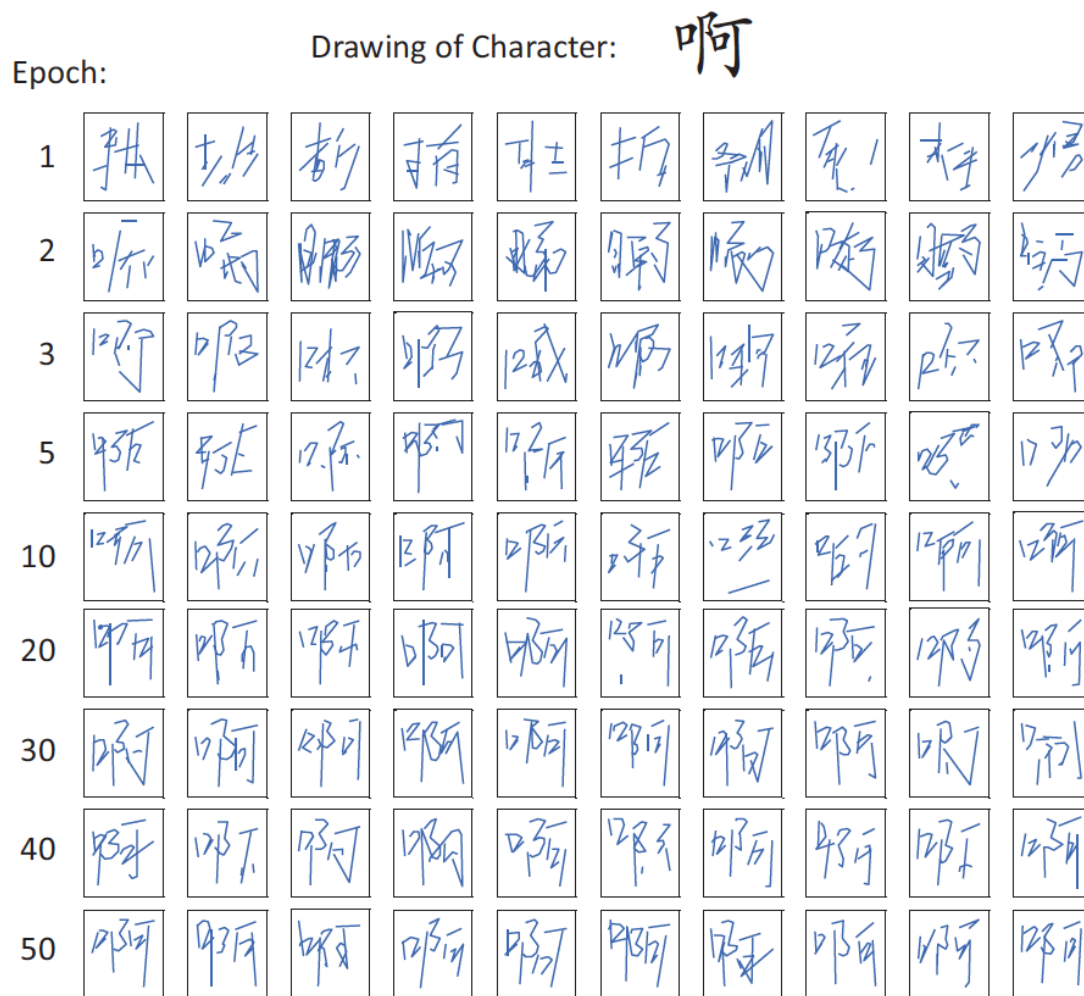
Generative RNN model: writing process

- Given a character index
- Choose its embedding vector $\mathbf{c}$
- Feed it into GRU
- Sampling a pen-movement from GMM $\mathbf{d}$
- Decide pen-state via softmax $\mathbf{s}$
- Update hidden state $\mathbf{h}$
- Feed $\mathbf{cdsh}$ to next step
- Continue writing until reach “end-of-char” state



Generative RNN model

- First few epochs, it is just random drawing
- After 30 epochs, the character can be correctly written



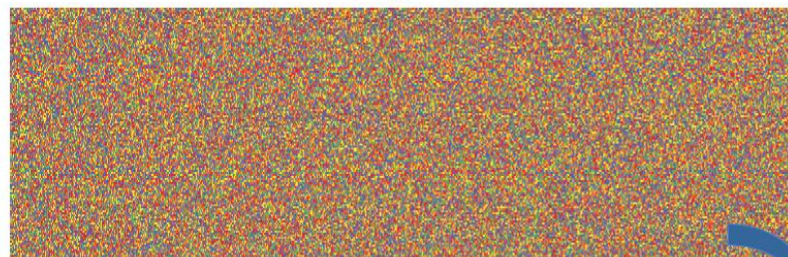
Character embedding matrix

- 3755 character embed in 500D space
 - a 500x3755 matrix
- Nearest neighbor via Euclidean distance:
 - Similar characters are grouped
 - Automatically find structure during writing

Note we did not provide similar-character info

This is automatically discovered during generation

character embedding matrix: 500×3755



10-nearest neighbors

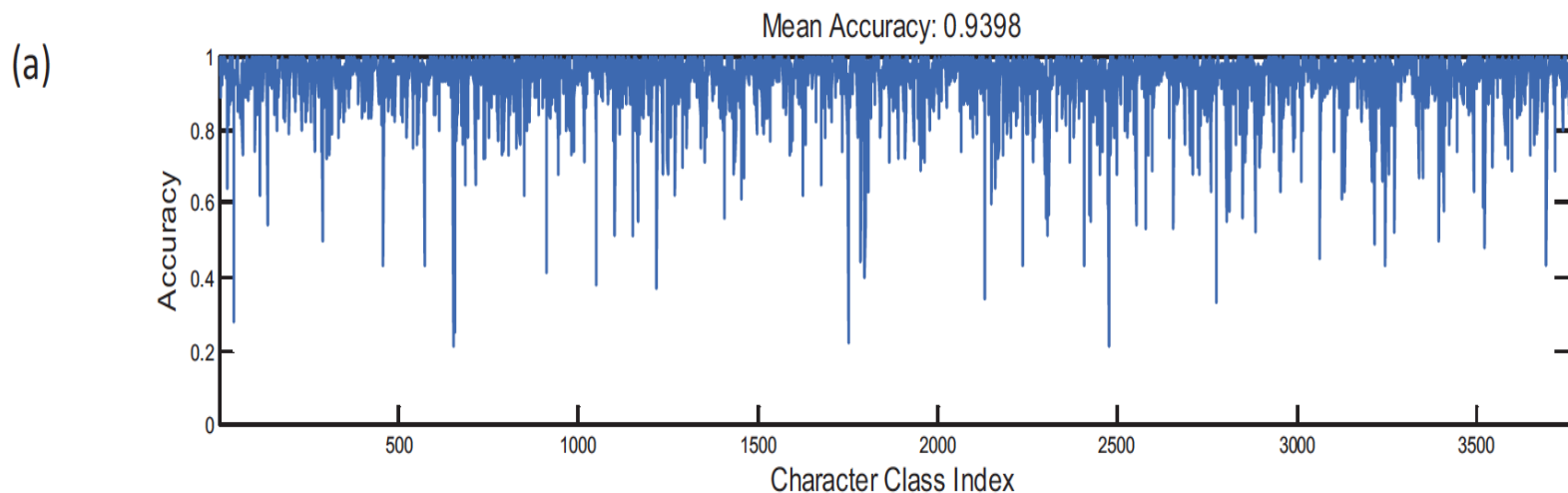
未	→	夫禾沫末昧扶木宋朱林
太	→	汰犬本酖桔杆扣尽村扶
天	→	无夫矢元秩大庆杆规扶
白	→	自百佰柏伯曰泊日倡拍
儿	→	几亿肌讥见吭元化肮无
己	→	已巳纪记巴兄尺几气弓
介	→	价乔阶疥芥齐午各而分
抱	→	泡胞咆炮跑抢饱袍鲍枪
比	→	此批化北砒庇毗抢讹混
成	→	咸城诚喊碱戌感减绒栈

Generative RNN model: examples



- Write 3755 characters: Character embedding
- Correctly writing: Softmax pen-state
- Diversity in style: GMM pen-direction

Generative RNN model: examples



(b)

(c)

托 33% 托 托 托 托 托 托 托 托 托 托 托?

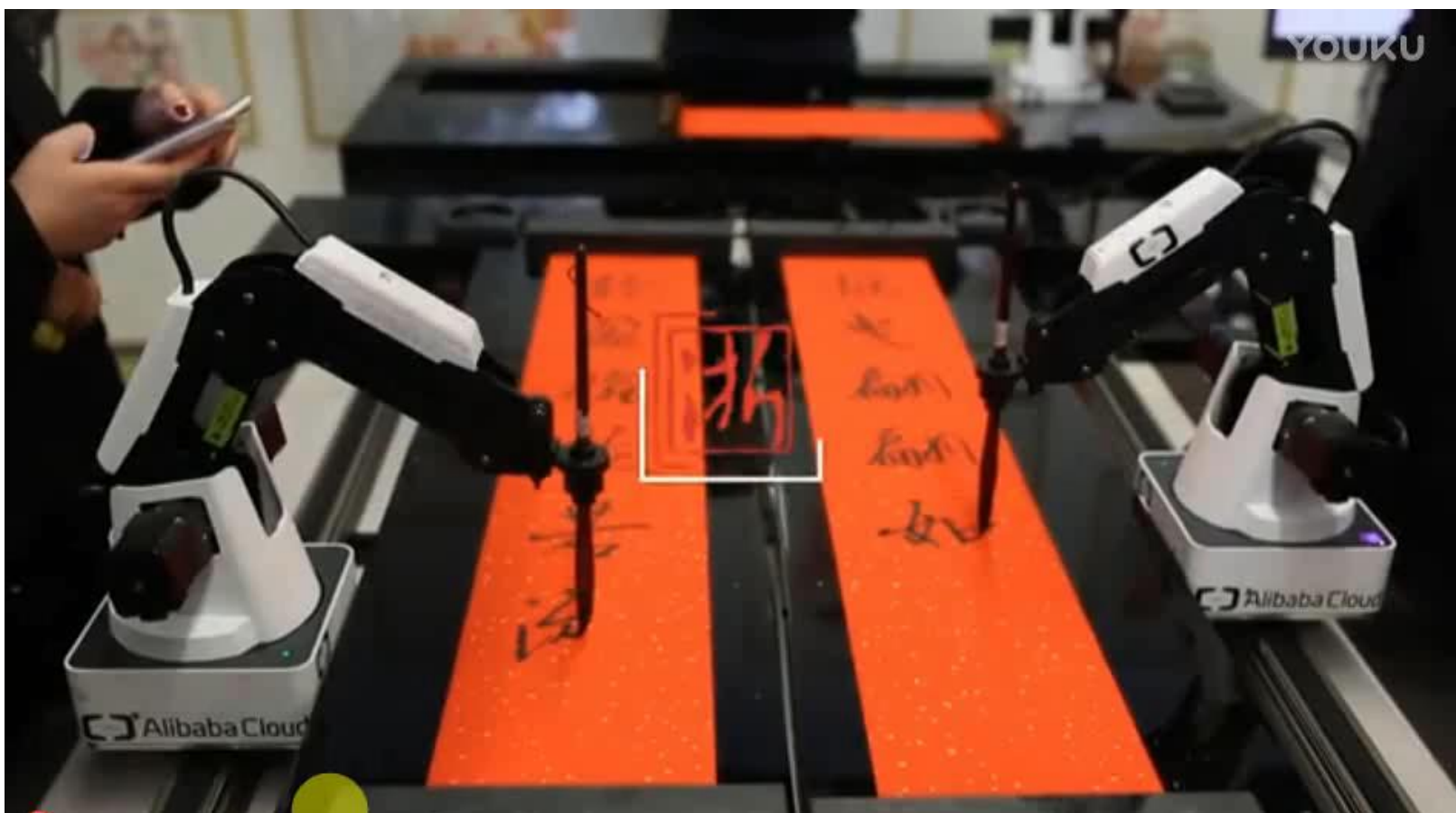
话 38% 话 话 话 话 话 话 话 话 话 话 话?

漫 44% 漫 漫 漫 漫 漫 漫 漫 漫 漫 漫 漫?

咨 100% 咨 咨 咨 咨 咨 咨 咨 咨 咨 咨 咨

缀 100% 缀 缀 缀 缀 缀 缀 缀 缀 缀 缀 缀

酌 100% 酌 酌 酌 酌 酌 酌 酌 酌 酌 酌 酌



致谢

- PPT由向世明、张煦尧老师提供

Thank All of You!
(Questions?)